DESIGN AND DEPLOYMENT OF A SECURE CLOUD-BASED PATIENT DATA MANAGEMENT SYSTEM USING A THREE-TIER ARCHITECTURE ON AWS

MOHAMED OMAR MOHAMED GAD MAKHLOUF

UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI TEKNOLOGI MALAYSIA
DECLARATION OF THESIS

Author's full name	:		
Matric No.	:	Academic Session	:
Date of Birth	:	UTM Email	:
Thesis Title	:		

I declare that this thesis is classified as:

☐ **OPEN ACCESS** — I agree that my report to be published as a hard copy or made available through online open access.

☐ **RESTRICTED** — Contains restricted information as specified by the organization/institution where research was done. *(The library will block access for up to three (3) years)*

☐ **CONFIDENTIAL** — Contains confidential information as specified in the Official Secret Act 1972)

(If none of the options are selected, the first option will be chosen by default)

I acknowledged the intellectual property in the thesis belongs to Universiti Teknologi Malaysia, and I agree to allow this to be placed in the library under the following terms :

1. This is the property of Universiti Teknologi Malaysia
2. The Library of Universiti Teknologi Malaysia has the right to make copies for the purpose of only.
3. The Library of Universiti Teknologi Malaysia is allowed to make copies of this thesis for academic exchange.

Signature of Student:

Signature :

Full Name
Date :

Approved by Supervisor(s)

Signature of Supervisor I:	Signature of Supervisor II
Full Name of Supervisor I	Full Name of Supervisor II
Date :	Date :

NOTES : If the thesis is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restriction

This letter should be written by a supervisor and addressed to Perpustakaan UTM. A copy of this letter should be attached to the thesis.

Date:

Librarian

Jabatan Perpustakaan UTM,

Universiti Teknologi Malaysia,

Johor Bahru, Johor

Sir,

CLASSIFICATION OF THESIS AS RESTRICTED/CONFIDENTIAL

TITLE:

AUTHOR'S FULL NAME:

Please be informed that the above-mentioned thesis titled _____ should be classified as RESTRICTED/CONFIDENTIAL for a period of three (3) years from the date of this letter. The reasons for this classification are

(i)

(ii)

(iii)

Thank you.

Yours sincerely,

SIGNATURE:

NAME:

ADDRESS OF SUPERVISOR:

"We hereby declare that we have read this thesis and in our opinion this thesis is sufficient in terms of scope and quality for the award of the degree of Bachelor of Computer Science (Computer Networks and Security) in Specialization"

Signature	:	
Name of Supervisor I	:	JOHAN MOHAMAD SHARIF
Date	:	MAY 30, 2026
Signature	:	
Name of Supervisor II	:	SECOND SV
Date	:	MAY 30, 2026
Signature	:	
Name of Supervisor III	:	THIRD SV
Date	:	MAY 30, 2026

Declaration of Cooperation

This is to confirm that this research has been conducted through a collaboration _____ and _____.

Certified by:

Signature :
Name :
Position :
Official Stamp
Date

* This section is to be filled up for theses with industrial collaboration

Pengesahan Peperiksaan

Tesis ini telah diperiksa dan diakui oleh:

Nama dan Alamat Pemeriksa Luar **:**

Nama dan Alamat Pemeriksa Dalam **:**

Nama Penyelia Lain (jika ada) **:**

Disahkan oleh Timbalan Pendaftar di Fakulti:

Tandatangan :

Nama :

Tarikh :

DESIGN AND DEPLOYMENT OF A SECURE CLOUD-BASED PATIENT DATA MANAGEMENT SYSTEM USING A THREE-TIER ARCHITECTURE ON AWS

MOHAMED OMAR MOHAMED GAD MAKHLOUF

A thesis submitted in fulfilment of the
requirements for the award of the degree of
Bachelor of Computer Science (Computer Networks and Security)

Computer Science
Computing
Universiti Teknologi Malaysia

MAY 2026

DECLARATION

I declare that this thesis entitled *"DESIGN AND DEPLOYMENT OF A SECURE CLOUD-BASED PATIENT DATA MANAGEMENT SYSTEM USING A THREE-TIER ARCHITECTURE ON AWS"* is the result of my own research except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature	:	_____
Name	:	MOHAMED OMAR MOHAMED GAD MAKHLOUF
Date	:	MAY 30, 2026

ACKNOWLEDGEMENT

I dedicate this template to those who have spent countless hours to make this possible.

ABSTRACT

In this study, a design of the Secure Cloud-Based Patient Data Management System for the Alamin Clinic of Saudi Arabia, a privately owned healthcare organization, is introduced. This private healthcare facility faced the challenge of having its on-premise infrastructure breached by a ransomware that led to the encryption of all patient data and operational disruptions. Four major vulnerabilities that have allowed such cyber-attack are identified in this study. They include a flat network architecture; manual infrastructure management ; lack of data encryption, and a lack of infrastructure recovery plan. Thus, these factors enabled the ransomware attack. They are also reflect the security posture of other similar small scale private healthcare organizations that use their legacy on-premises infrastructure. In order to address these challenges, the proposed design uses a three-tier architecture hosted within the environment of Amazon Web Services, where presentation, application, and database layers are placed in separate subnets within a VPC and database tier has no Internet access paths. In addition, a solution with Infrastructure as Code is used using Terraform. It allows not only an automated process of deployment but also quick recovery in case of infrastructure crashes. The DevSecOps continuous integration and delivery pipeline uses Trivy, SonarQube, and Checkov applications and provides automated vulnerability scan at each code commit, making any critical finding impossible to deploy. Finally, the access to the application and database is restricted using Identity and Access Management policies and Role Based Access Control. Row-level security of the PostgreSQL databases ensures the enforcement of individual data isolation at the storage layer. The proposed network topology, security group configuration, database schema with row-level security policies, DevSecOps pipeline specifications, and wireframe for interfaces are included in this design to establish an architectural blueprint. The main evaluation criteria are set as a recovery time objective under fifteen minutes and achieving passing HIPAA security posture, measured through AWS Security Hub.

ABSTRAK

Dalam kertas penyelidikan ini, reka bentuk Sistem Pengurusan Data Pesakit Berasaskan Awan yang Selamat untuk Klinik Alamin di Arab Saudi, sebuah organisasi penjagaan kesihatan swasta, diperkenalkan. Fasiliti ini menghadapi cabaran pencerobohan infrastruktur on-premise oleh perisian tebusan yang mengakibatkan penyulitan data pesakit dan gangguan operasi. Empat kerentanan utama dikenal pasti: seni bina rangkaian rata, pengurusan infrastruktur manual, ketiadaan penyulitan data, dan ketiadaan pelan pemulihan. Faktor ini membolehkan serangan tersebut dan mencerminkan tahap keselamatan organisasi kesihatan kecil lain yang menggunakan infrastruktur warisan. Bagi menangani cabaran ini, reka bentuk cadangan menggunakan seni bina tiga lapis pada Amazon Web Services, di mana lapisan persembahan, aplikasi, dan pangkalan data diasingkan dalam subnet VPC tanpa capaian Internet bagi pangkalan data. Infrastruktur sebagai Kod menggunakan Terraform membolehkan pelaksanaan automatik dan pemulihan pantas. Talian paip DevSecOps menyepadukan Trivy, SonarQube, dan Checkov untuk imbasan kerentanan automatik pada setiap commit bagi menghalang penggunaan kod berisiko. Capaian dihadkan melalui polisi IAM dan RBAC, manakala keselamatan peringkat baris PostgreSQL memastikan pengasingan data pada lapisan storan. Topologi rangkaian, konfigurasi kumpulan keselamatan, skema pangkalan data, spesifikasi talian paip DevSecOps, dan rangka dawai antara muka disertakan sebagai pelan tindakan pelaksanaan PSM2. Kriteria penilaian utama adalah objektif masa pemulihan bawah lima belas minit dan pencapaian tahap keselamatan HIPAA melalui AWS Security Hub.

TABLE OF CONTENTS

REFERENCES **87**

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AES	-	Advanced Encryption Standard
ALB	-	Application Load Balancer
API	-	Application Programming Interface
AWS	-	Amazon Web Services
AZ	-	Availability Zone
CI/CD	-	Continuous Integration and Continuous Delivery
CIDR	-	Classless Inter-Domain Routing
CVE	-	Common Vulnerabilities and Exposures
DevSecOps	-	Development, Security, and Operations
EBS	-	Elastic Block Store
ECR	-	Elastic Container Registry
EHR	-	Electronic Health Record
ER	-	Entity-Relationship
FHIR	-	Fast Healthcare Interoperability Resources
FR	-	Functional Requirement
FTP	-	File Transfer Protocol
HCL	-	HashiCorp Configuration Language
HIPAA	-	Health Insurance Portability and Accountability Act
HL7	-	Health Level 7
HMS	-	Hospital Management System
HTTPS	-	Hypertext Transfer Protocol Secure
IAM	-	Identity and Access Management
IaC	-	Infrastructure as Code
JWT	-	JSON Web Token
KMS	-	Key Management Service
NACL	-	Network Access Control List
NAT	-	Network Address Translation

NFR	-	Non-Functional Requirement
NVD	-	National Vulnerability Database
PC	-	Personal Computer
PDMS	-	Patient Data Management System
PDPL	-	Personal Data Protection Law
PSM	-	Projek Sarjana Muda
RBAC	-	Role-Based Access Control
RDS	-	Relational Database Service
RLS	-	Row-Level Security
RTO	-	Recovery Time Objective
SAST	-	Static Application Security Testing
SHA	-	Secure Hash Algorithm
SQL	-	Structured Query Language
SSL	-	Secure Sockets Layer
TLS	-	Transport Layer Security
UAT	-	User Acceptance Testing
UC	-	Use Case
UML	-	Unified Modelling Language
US	-	User Story
UTM	-	Universiti Teknologi Malaysia
UUID	-	Universally Unique Identifier
VPC	-	Virtual Private Cloud
YAML	-	YAML Ain't Markup Language

LIST OF APPENDICES

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid adoption of digital technologies has transformed the entire process of collecting and storing information in the clinic. Transitioning from paper-based processes to the use of EHRs in clinics and hospitals which made healthcare information management infrastructure crucial since sensitive patient data must be protected. Information security and protection of secret patient information has always been an area of concern, especially in cases of breaches where patient privacy and security is put at risk.

While the importance of adopting a secure infrastructure becomes obvious with the growth in the number of cybersecurity threats, there are still many small clinics and hospitals which rely on the legacy on-premises server solution and ignore the growing risks linked with the lack of enough cybersecurity measures. The traditional server infrastructure does not allow implementing any additional measures to prevent attacks or limit their impact on the system, thus increasing the risks Significantly.

Thus, this project aims at designing and implementing a cloud-based solution that would be able to effectively protect patients' confidential information from malicious attacks and unauthorized access. The proposed cloud-based system will feature a three-tier architecture and be deployed into isolated network subnets using Virtual Private Cloud (VPC) and Infrastructure as Code (IaC) solutions such as Terraform. The DevSecOps pipeline will enable developers to scan every piece of code before deployment.

A practical example of the need for this system is seen at the case of the Alamin Clinic which suffered a cyberattack and became a victim of ransomware.

Thus, the selected organization will provide requirements for the system design and implementation to solve the identified problem.

1.2 Problem Statement

At present, the management of the patient data management system at the Alamin Clinic relies on an on-premises physical server that provides services of the web frontend, application backend, and patient database in a single network without any segmentation of layers of the architecture. In other words, all of these elements of infrastructure are manually configured, deployed, and managed; for example, the code for applications is deployed to a production environment through FTP or USB drive while updates for antivirus programs and firewalls are applied on a manual basis with significant delays.

However, such an approach revealed its flaws when the clinic was hacked and its patients' data was affected by the ransomware threat. The hackers gained access to the system, Found its way into the network and encrypted the entire database. Due to the lack of network segmentation and isolation, all the components of the system became vulnerable and unusable until the data is restored manually.The lack of automation and regular backups increased downtime and made it impossible to restore data at time.

It appears that the problem described above align with a wider problem that affects small healthcare organizations. Argaw *et al.* (2019) noted that cyberattacks in hospitals take advantage of old infrastructure, poor network segregation, and inadequate patching policies. Moreover, as Al-Issa *et al.* (2019) reported, cloud computing technologies used in the healthcare industry have some specific problems related to access management and Unprotected date, among others.

In summary, the problems described above reveal one major gap typical for small scale organizations that is a lack of proper information security measures implemented since the beginning. To address the issue, the proposed solution will be based on cloud computing and ensure security from the very beginning through Compliance with the security-by-design principle.

1.3 Project Aim

The aim of this project is to design and deploy a secure cloud-based Patient Data Management System for Alamin Clinic using a three-tier architecture on AWS, Integrating Infrastructure as Code and a DevSecOps pipeline to ensure automated, reproducible, and security-validated deployments.

1.4 Project Objectives

The objectives of the project are:

(a) To investigate core concepts including cloud computing security, three-tier architecture, network segmentation, Identity and Access Management, and secure system design practices within the context of healthcare applications.

(b) To develop a secure Cloud-Based Patient Data Management System based on a three-tier architecture on AWS, covering Virtual Private Cloud networking, access control through IAM and Role-Based Access Control (RBAC), and a DevSecOps CI/CD pipeline with integrated security scanning.

(c) To evaluate the system's performance through automated vulnerability scanning using Trivy, SonarQube, and Checkov. Evaluation includes RTO stress testing via a simulated ransomware recovery scenario and a HIPAA compliance assessment using AWS Security Hub. Finally, User Acceptance Testing (UAT) will be conducted with Doctors, Admins, and Patients to validate functional requirements.

1.5 Project Scope

This project focuses on the design and deployment of a Secure Patient Data Management System for use by doctors, administrative staff, and patients at Alamin Clinic. The system will be deployed on AWS as a functional prototype, with security monitoring through scan reports, CloudWatch logs, and recovery time test results.

1.5.1 In-Scope

The following areas are within the scope of this project:

(a) Core patient record management functionality, including Patient Registration, Medical Records Management, Appointment Scheduling, and User Authentication with Role Management (Doctor, Admin, Patient).

(b) Design of a Virtual Private Cloud (VPC) with public subnets for public internet and private subnets for the application and database layers.

(c) Three-tier architecture Consists of a React frontend layer, a Node.js/Express backend layer on EC2, and a PostgreSQL database layer on Amazon RDS.

(d) AWS services including VPC, EC2, RDS, Application Load Balancer (ALB), NAT Gateway, and Internet Gateway.

(e) Network security controls including Security Groups and Network Access Control Lists (NACLs).

(f) Identity and Access Management (IAM) with least-privilege policies and Role-Based Access Control (RBAC) applied across all three user roles.

(g) Data encryption at rest using AES-256 through AWS Key Management Service (KMS) and encryption in transit using TLS/HTTPS.

(h) A DevSecOps CI/CD pipeline built on GitHub Actions with Docker containerization and Terraform IaC, incorporating automated security scanning using Trivy (container images), SonarQube (static code analysis), and Checkov (IaC misconfiguration scanning).

(i) Monitoring and audit logging using Amazon CloudWatch and AWS CloudTrail.

(j) HIPAA compliance posture assessment using AWS Security Hub.

1.5.2 Data and Subjects

The system will be evaluated using a pilot dataset of simulated patient records generated for testing purposes. User Acceptance Testing will be conducted with a minimum of three representative participants, one per user role (Doctor, Admin, Patient). No real patient data will be used during development or testing.

1.5.3 Out-of-Scope

To maintain focus on the security architecture and infrastructure, the following modules are explicitly excluded from this project:

(i) hospital billing and insurance claim management; (ii) pharmacy inventory and dispensing management; (iii) Emergency Room (ER) management; and (iv) specialized Obstetrics and Gynaecology (O&G) clinical modules.

1.6 Project Importance

The importance of the project comes out in several aspects: clinical, technical, and academic.

First of all, when it comes to clinical issues, one should understand that patient information is considered to be one of the most sensitive types of personal data. An information leak or loss of availability caused by a ransomware attack like in Alamin Clinic's case may have serious consequences, both for patients and doctors, as well as reputational and legal ramifications. Therefore, the development of a system that will be characterized by high availability, security, and integrity of patient data cannot be regarded as a technical task.

Secondly, one needs to consider the technical part of the project. Infrastructure as Code and DevSecOps approaches allowed for the conversion of an outdated and manual environment into a self healing infrastructure with the use of code and security validation. The whole infrastructure is coded using the Terraform language, and this gives us the opportunity to redeploy it from a clean state in minutes. Furthermore, a security scan conducted prior to deployment ensures that any threats are discovered and prevented from moving forward.

Finally, On the academic aspect. In this case, the project allows for practical application of the theoretical notions in terms of cloud security. Using HIPAA as the standard of compliance and AWS Security Hub as an evaluation tool.

1.7 Report Organization

This report is organized as follows:

Chapter 1 — Introduction presents the background to the project, the problem statement at Alamin Clinic, the project aim and objectives, scope, and the importance of the study.

Chapter 2 — Literature Review presents the background literature relevant to the project. Analysis of the current system at Alamin Clinic, a comparison of existing healthcare management systems, and a review of the technologies and methods used in the proposed solution, including cloud security frameworks, three-tier architecture, Infrastructure as Code, and DevSecOps practices.

Chapter 3 — System Development Methodology presents the development approach adopted for this project. It justifies the choice of methodology, outlines its phases as applied to this system, describes the technologies used, and presents the system requirements analysis.

Chapter 4 — Requirement Analysis and Design presents the detailed functional and non-functional requirements of the system, followed by the complete system design including the network architecture, database schema, IAM policy structure, and system interface design.

Chapter 5 — Conclusion summarizes the achievement of the project objectives, reflects on the limitations of the current implementation, and proposes directions for future improvement.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Developing a secure cloud-based system that manages patients' information require knowledge from various areas, which involve threats faced by healthcare information systems, architecture principles used in cloud computing, patients' data regulations, and DevSecOps processes that ensure that the system is developed and deployed in a secure manner. This chapter reviews the knowledge on how the proposed system can be designed are reviewed. This is done through first considering the case study organization before moving to general technological and literature.

Section 2.2 introduces the case study organization, Alamin Clinic, through analysing its organizational structure, current manual operating model, and the interview outcomes from key people in clinic. The section also analyzes the difference between the current followed process and the process followed when the new system is introduced in the clinic. Section 2.3 carries out a 5W 1H analysis on the problems faced by the current system. Section 2.4 compares the proposed system to existing healthcare systems, including the unique aspects from each existing healthcare management system. Section 2.5 highlights the major technologies used in designing the new system and their supporting literature. Section 2.6 presents key conclusions from the literature review.

2.2 Case Study: Alamin Clinic (Saudi Arabia)

Alamin Clinic is a private independent health care service in Saudi Arabia. In essence, this is the organization to be considered a stakeholder and used as the main case study for this proposal. The incident that has happened to Alamin Clinic and led to the development of this idea was an attack by ransomware on their IT services.

The patient base of Alamin Clinic includes people who seek various types of consultations, follow-up sessions, and health record administration services. The key stakeholders, which include doctors, administrative staff, and patients themselves, all have different ways of interacting with the database of patients in the clinic, and therefore, will need different access control mechanisms. this diversity of roles makes the design of a Role-Based Access Control (RBAC) model a fundamental requirement of the proposed system.

2.2.1 Alamin Clinic: Organizational Structure

Alamin Clinic has a three-level organizational hierarchy that is usually common among small private clinics in the region.

The clinical level consists of general specialist doctors who consult patients, diagnose illnesses, and keep medical records. Doctors need read and write permissions on the medical records they treat. They should not be able to access administrative records like bills and salaries of employees.

The administrative level is made up of people who register and schedule patients. Administrators should only be able to access registration data and appointment data but should not be able to see anything else like medical records. Separation of responsibilities is crucial for patient confidentiality and should be guaranteed by the IAM structure proposed here.

Patients registered at Alamin Clinic can log into the application and check their medical records and scheduled appointments. Patient access will only allow them to read their records but not any other data like other patients' records or any administrative record.

The clinic also has a basic IT function that had only on-premise servers. There was no security policy or any patch management policy before the proposed migration.

The organizational structure is illustrated in Figure 2.1.

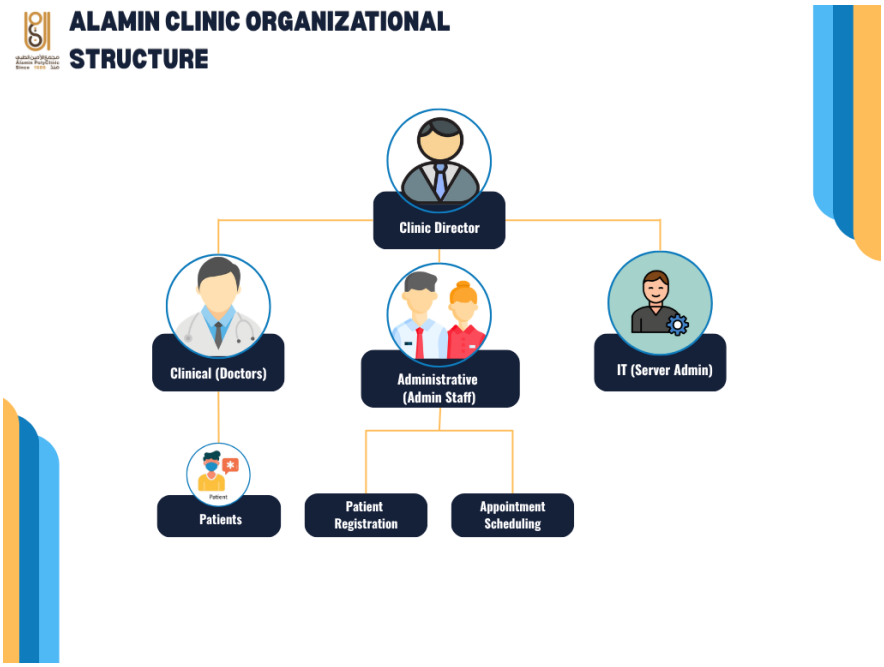


Figure 2.1 Alamin Clinic Organizational Structure

2.2.2 Manual Operation

The existing system used at Alamin Clinic for storing patient information follows a traditional on-premises design model defined by the following characteristics: manual server configuration, manual deployment, local hosting, and reactive security.

Manual Server Configuration. The server hardware installed at the clinic is configured locally by IT staff of the clinic. Server configuration is done manually without maintaining infrastructure versions and automating server deployment processes. This methodology is Unreliable because it relies on manual configuration of servers with no control over drift. It is impossible to roll back the configuration in case of failure because no automated server provisioning is used (Paidy and Chaganti, 2024).

Manual Deployment. The code deployed in the development environment is manually moved to the production server with the use of either FTP clients or storage devices, such as USB drives. This methodology slowdown the deployment time and prevents implementing automatic checks of the code for security or quality issues before deploying it to the production environment. Furthermore, the manual

deployment process ensures that malware can enter the production environment bypassing any controls.

Local Hosting. The frontend of the website, the backend logic of the application, and the patient database are all hosted on the same physical servers. There is no network segmentation between these layers, which means that gaining access to any of the components is enough to access the rest. The absence of network segmentation and isolation is considered one of the key factors that contribute to ransomware infections among healthcare facilities, allowing the malicious code to move directly to the database layer (Argaw *et al.*, 2019; Thamer and Alubady, 2021).

Reactive Security. Firewalls and antivirus solutions are updated manually by IT staff according to available information about existing threats. This practice does not provide enough protection against emerging cyberattacks because updates are carried out reactively and are therefore lagging behind the threat landscape. As a result, known vulnerabilities are patched late, exposing the facility to attacks.

In effect, it was shown by the fact that the clinic's security was compromised after a ransomware attack, where the hackers managed to encrypt all the patients' information, making the records inaccessible for use by the clinic. As such, it was evident that there were no protective measures to block an intruder from reaching their target data.

2.2.3 Interview with Clinic Stakeholders

In order to get first hand information about the problems faced by the clinic and weaknesses of the system, two major stakeholders of the clinic were interviewed in person through a video conferencing. These include one Administrative Staff member who is involved in registering patients and scheduling appointments and a Clinical Staff member who performs routine operations at the clinic. Considering the privacy concerns of the participants, there was no audio recording of the interviews; instead, notes were made while conducting the interviews. There were a total of eight open-ended questions in the interview protocol.

The following questions were used to guide each interview:

(a) How do you currently access and manage patient records on a daily basis?

(b) What are the most significant difficulties you face when using the current system?

(c) How was the ransomware incident discovered, and what was the immediate impact on your work?

(d) How long was the clinic unable to access patient records, and how was this period managed?

(e) What data was lost or inaccessible as a result of the attack?

(f) Were there any security policies or backup procedures in place at the time of the incident?

(g) What features of a new system would most improve your day-to-day work?

(h) What security or privacy concerns do you have about moving patient data to a cloud system?

Table 2.1 Summary of Stakeholder Interview Findings and System Implications

Question Theme	**Administrative Staff**	**Clinical Staff**	**System Design Implication**
Daily Access	Individual logins for a 3rd-party desktop app via external vendor. Limited remote access.	Same desktop app; managed by provider. No in-house IT capability for diagnosis.	Transition to a web-based React interface. Remote access supported by default via browser.

Question Theme	**Administrative Staff**	**Clinical Staff**	**System Design Implication**
Main Difficulties	No unique patient IDs cause duplication. Slow billing. Manual Excel tracking for appointments.	Inconsistent entry compounding duplication. No automated monitoring or alerting for the server.	UUIDs enforced at DB level. Integrated scheduling module replaces Excel. CloudWatch for alerting.
Ransomware Discovery	All data inaccessible. Operations halted. Used paper forms as fallback.	All departments frozen simultaneously. Doctors could not access files; appointments cancelled.	Private subnet isolation for DB. Security Groups and NACL rules enforce strict tier-based blocking.
Downtime Duration	72 hours of total blackout. Restoration failed due to corrupted or incomplete data.	5-day recovery window. Significant backlog accumulated before operations resumed.	Terraform-based IaC allows full rebuild in < 15 mins. Independent RDS snapshots.
Data Lost	All patient records inaccessible. No clean backup existed to restore the system.	Records in the attack period were permanently lost. Access unavailable during recovery.	Daily RDS snapshots with point-in-time recovery. Multi-AZ standby replica for high availability.
Security Measures	Basic antivirus with manual updates. No formal backup policy or scheduled reviews.	No formal security policy or staff awareness of recovery procedures.	Security Hub continuous HIPAA monitoring. IAM and NACL configurations version-controlled in Terraform.

Question Theme	Administrative Staff	Clinical Staff	System Design Implication
Improvements	Web interface, auto-notifications, remote access, unique IDs, and a public website.	Dept-specific modules, automated workflows, and faster performance during consultations.	Role-based dashboards in React. Patient UUIDs for clean registration. Public site is future scope.
Migration Concerns	Data sovereignty and internet reliability in the clinic's operating region.	Data privacy, access control clarity, and training requirements for the new system.	Hosting in AWS Singapore. Row-Level Security ensures users only access authorised records.

The results of the interview confirm the four operational issues found during the interview analysis discussed in Section 2.2.2. However, it brings out three new dimensions to the problem. Firstly, the human frustration on the organization taken by the ransomware attack a five days of operational interruption and full shutdown of clinic operation; secondly, the lack of security policy knowledge and backup practices on the part of operational staff and thirdly, the data quality issue associated with the difficulty to link patients to unique ID numbers.

2.2.4 Current System Workflow

The current process for managing patient information in Alamin Clinic relies entirely on manual handling without any automated means. Figure 2.2 highlights the flowchart for managing the main processes for patient information from the arrival of patients to storing their records.

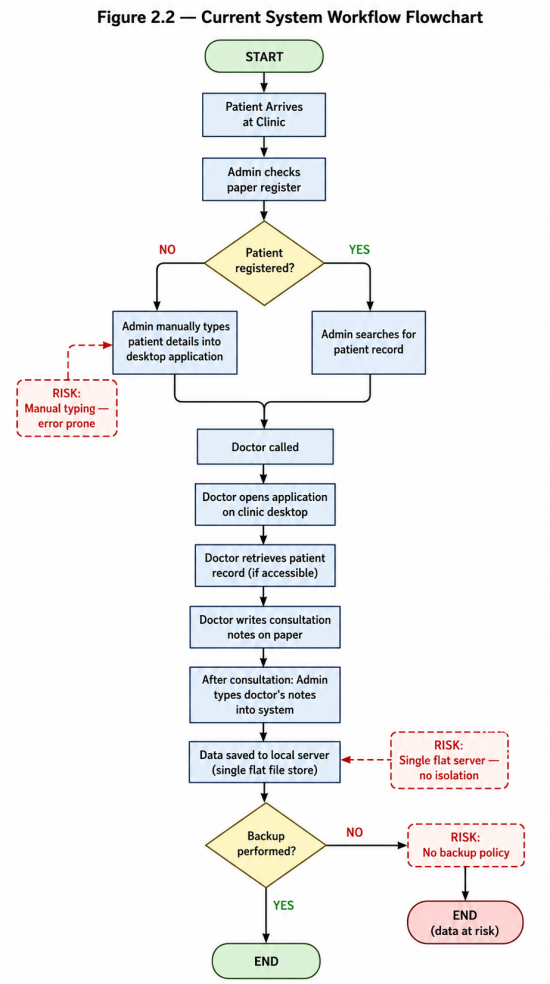


Figure 2.2 Current System Workflow Flowchart

There are five significant threats associated with the current workflow process. Firstly, all inputs into the process are made manually by administrative staff, resulting in errors and time wastage. Secondly, there is no distinction between the application that processes the information and the database that holds the data because both use the same server process and file structure. Thirdly, there is no distinction between the logins for physicians and administrative staff since both log into the same application with the security measures depending purely on trust. Fourthly, there is no automatic backup system for the process, meaning that all information storage depends entirely on the availability of the actual server. Lastly, there is no way to track who accessed or changed what in patients' files.

2.2.5 Proposed System Workflow

The proposed system replaces each manual, insecure step in the current workflow with an automated, security-enforced equivalent. Figure 2.3 presents the flowchart of the proposed system's patient record management process.

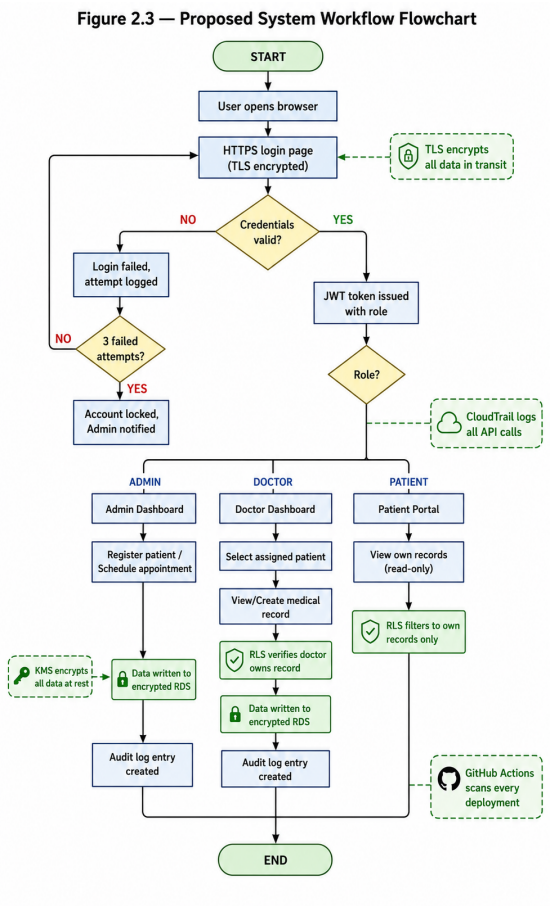


Figure 2.3 Proposed System Workflow Flowchart

The solution presented here solves all of the mentioned vulnerabilities in the system. Instead of manual data entry, there is now an input-validation web page. In addition, the single-server design is swapped for a three-tier VPC architecture in which the database has been separated into a private subnet that does not have internet access. Instead of using role enforcement, the authentication mechanism relies on JWT and PostgreSQL row-level security to control who accesses what data. Physical hardware backup is no longer used; instead, the system will automatically create snapshots in

RDS and redeploy the entire infrastructure with Terraform. Finally, audit logging has been introduced via CloudTrail and an audit table in the application.

Table 2.2 presents a side-by-side comparison of the current and proposed workflows across the five identified vulnerability dimensions.

Table 2.2 Comparison Between the Current System and the Proposed Cloud-Based System

Dimension	**Current System**	**Proposed System**
Data entry	Manual typing by admin, error-prone	Structured web forms with validation
Architecture	Single flat server, all tiers co-hosted	Three-tier VPC — presentation, app, DB isolated
Authentication	Single shared login, trust-based	JWT tokens + RBAC enforced at app & DB layers
Backup and recovery	No backup policy; full data loss risk	Automated RDS snapshots + Terraform redeploy in ¡ 15 min
Audit trail	None	CloudTrail API logs + application audit_log table
Deployment	Manual FTP/USB, no security gate	GitHub Actions CI/CD with Trivy, SonarQube, & Checkov
Encryption	None at rest; inconsistent in transit	KMS AES-256 at rest; TLS 1.2+ in transit

2.3 Current System Analysis

2.3.1 Problem Analysis Using the 5W 1H Framework

The 5W 1H framework — Who, What, When, Where, Why, and How — provides a structured approach that ensures all dimensions of the issue are addressed

before a solution is designed. Table 2.3 applies this framework to the Alamin Clinic problem.

Table 2.3 5W1H Analysis of the Current System Vulnerabilities

Dimension	Analysis
Who is affected?	Doctors cannot access patient records or create diagnoses. Administrative staff cannot manage appointments. Patients cannot receive care during outages. The clinic director faces operational and potential legal liability for data loss.
What is the problem?	A ransomware attack encrypted the patient database. No recovery capability or backups existed. The systemic problem is an infrastructure designed without security controls at any layer.
When does it exist?	At all times. The server is exposed to the internet with no isolation. Patches are reactive, and every manual deployment is a security risk.
Where does it originate?	In the flat network architecture of the on-premise server, providing no barrier to lateral movement. In the absence of network segmentation between web, app, and database layers.
Why does it persist?	Small healthcare providers lack the budget for enterprise security platforms and under-resourced IT staffing. This is the market gap the proposed system addresses.
How will it be solved?	Through a security-by-design cloud architecture: (1) Three-tier VPC isolation. (2) Terraform IaC for configuration consistency. (3) DevSecOps pipeline (Trivy, SonarQube, & Checkov). (4) IAM/RBAC with PostgreSQL security. (5) KMS encryption & TLS in transit. (6) AWS Security Hub for HIPAA compliance.

2.3.2 Security Domain Analysis

The assessment of the existing infrastructure of the Alamin Clinic, before Starting on the migration process, shows that there are weaknesses in the four areas of security:

Network Architecture. In the case of the clinic, the network architecture is highly simplified in that all three tiers of applications—the web, application, and

database are hosted in one machine with one network interface, No DMZ for internet connected parts of the network, and no way to prevent lateral movements once the system is breached. This completely violates the concept of defense in depth because it does not offer more than one line of defense (Al-Issa *et al.*, 2019).

Access Control. The user access to the system is determined based on the authentication carried out only at the application level without having any kind of identity management. There are role separations between doctors, administration, and patients that are implemented in the application level but do not have any restrictions at the networking or storage level.

Data Security. The patient data stored in the database of the clinic is not encrypted in its static form. The transmission of the data from the browser of the user to the server is not reliably encrypted using TLS due to the lack of analysis of the HTTP settings used by the clinic. Both the static and the dynamic data are exposed to the threat of being intercepted and stolen(Shojaei *et al.*, 2024).

Operational Resilience. There is no official data backup policy at the clinic, nor is there any officially documented way to recover from such situations. This scenario has been noted in many studies involving ransomware attacks on other small health care providers.Argaw *et al.* (2019) noted that the lack of backup and recovery policy contributed to prolonged disruption during such incidents.

In combination, all of the flaws above shows that the existing model does not meet even the basic criteria of security for a healthcare information system. As such, there is a clear need for a complete overhaul of the system architecture to improve its security level.

2.4 Comparison between existing systems

To frame the proposed model among other solutions for managing healthcare, a comparative table is presented below that shows how the proposed solution fits alongside three other systems, namely, a conventional HMS system, the OpenEMR system, the Epic Systems solution, and the proposed Secure Cloud PDMS.

Table 2.4 Comparison of Existing Healthcare Management Systems

System	**Key Features**	**Security Model**	**Main Limitations**
Traditional On-Premise	Full data control, no internet dependency.	Reactive — manual patches, perimeter firewall.	High ransomware risk, no isolation, poor scalability.
OpenEMR	Patient portal, billing, EHR.	Reactive — community patches.	Manual security updates, no auto-deployment.
Epic Systems	Comprehensive clinical suite, HL7/FHIR.	Proactive — vendor-managed updates, IAM.	Prohibitive cost for small clinics, vendor lock-in.
Proposed System	Patient records, roles, cloud-native.	Proactive, shift-left — automated scanning, RBAC.	Infrastructure focus; billing/pharmacy out of scope.

The comparison highlights a unique weakness of the existing market towards small healthcare providers. On-premise software applications and open-source projects like OpenEMR provide high levels of flexibility but require all aspects of security management to be undertaken by the IT staff of the medical facility, as proven by the history of the Alamin Clinic. However, even though commercial products, including those by Epic Systems, provide an advanced level of security, they are too expensive and complicated for small private providers.

This project will fill this niche through a combination of a low-cost model associated with the use of cloud hosting and the application of automation technologies used in commercial enterprise-level systems. The inclusion of the infrastructure as code through Terraform and the automatic scanning of security issues during the CI/CD process will reduce the need for manual IT procedures, thus minimizing this primary weakness of the existing solution.

2.4.1 Features Adopted from Existing Systems

The comparison provided in Table 2.4 made it possible to choose the features from the existing systems, which would be relevant to the goals of the suggested system both in terms of scope and security considerations. The features borrowed from the existing systems and the modifications made are shown in Table 2.5.

Table 2.5 Features Adopted from Existing Systems

Feature	**Adopted From**	**Justification**	**Adaptation in Proposed System**
Patient Registration	OpenEMR	Core requirement; directly addresses ransomware data loss.	Implemented with PostgreSQL RLS enforcement.
Appointment Scheduling	OpenEMR	Operational necessity; appointment data was highly disruptive when lost.	Role-restricted access: Admin modifies, doctors/patients view only.
Patient Portal	OpenEMR	Reduces admin load; addresses patient autonomy.	Read-only access; accounts strictly created by admin.
RBAC Model	Epic Systems	Enterprise-grade pattern validated by Singh & Chatterjee (2015).	Three-layer enforcement: JWT, IAM policies, and PostgreSQL RLS.
Audit Logging	Epic Systems	HIPAA requirement; key missing finding from incident review.	Dual mechanisms: AWS CloudTrail and application audit_log table.

Continued on next page

Table 2.5 – Continued from previous page

Feature	**Adopted From**	**Justification**	**Adaptation in Proposed System**
HIPAA Framework	Epic Systems	Recognized benchmark for healthcare data security.	Continuous measurement via AWS Security Hub HIPAA standard.

The features not taken from any current technology are as follows: Billing for hospitals and insurance management (OpenEMR, Epic); Pharmacy management (Epic); ER management (Epic); and HL7/FHIR interoperability (Epic). These features are not incorporated into the scope of the suggested technology to narrow down the scope of the security framework. This is explained in chapter 1, section 1.5 (Project Scope).

Another difference of the system under discussion from its competitors lies in the comparative analysis of the standard AWS architecture design template. Table 2.6 compares these two types of templates.

Table 2.6 Proposed System vs Standard AWS Architecture

Feature	**Standard AWS Architecture**	**Proposed Solution**
Deployment	Manual or basic scripts	Fully automated via **Terraform IaC**
Security Model	Added after build (Reactive)	**Shift-left** — scanned at every CI/CD stage
Post-attack Recovery	Long manual rebuild process	**Instant redeployment** from Terraform state
Data Protection	Basic private subnets	Hardened **NACLs** + Private **RDS** + **KMS** encryption
Compliance	Ad hoc manual checks	HIPAA posture measured via **AWS Security Hub**

2.5 Literature Review of Technology Used

2.5.1 Cloud Computing in Healthcare

Since the early 2010s, the use of cloud computing technology within healthcare has been extensively researched. The study performed by Ahuja *et al.* (2012), which can be considered one of the first comprehensive investigations of cloud computing usage in healthcare settings, concludes that cost savings, scalability, and improved data access are the most common factors contributing to widespread adoption, whereas data protection, regulatory issues, and vendor reliability stand out as the key barriers. All those findings still hold true today. In particular, the security problems mentioned by Ahuja *et al.* (2012), such as concerns about data sovereignty and access control, have influenced the design of the IAM policy for our solution.

In the same spirit, Al-Issa *et al.* (2019) investigated cloud security issues specific to eHealth environments. Their survey revealed that unauthorized access, data corruption, and service availability issues are the leading threats affecting eHealth clouds. Accordingly, each threat mentioned in their study is addressed by the corresponding design requirement in our case: unauthorized access is controlled via IAM/RBAC; integrity violations are prevented using CloudTrail audit logging and KMS encryption; and availability is guaranteed with a multi-AZ architecture and fast recovery provided by Terraform.

The stakeholder concerns regarding cloud data privacy raised by the clinic stakeholders in section 2.2.3 interviews are handled by the proposed solution with the use of (a) the AWS region chosen to be in gulf region according to PDPL data residency rules, (b) IAM least privilege principles, and (c) row-level security of PostgreSQL database which ensures that one cannot access another user's data regardless of infrastructure level access(Ramayanam, 2025).

2.5.2 Three-Tier Architecture

The three-tier architectural approach involves breaking down a web application into three different layers which are physically and logically segregated from one another, namely the presentation tier (frontend), application tier, and data tier (database). There are two key advantages to using this model from a security perspective. First, the damage scope for a breach in a particular tier is significantly

reduced. Second, it allows for the independent scaling and access control to be implemented at the level of each individual tier.

In this particular case, the three-tier architecture is deployed using the AWS Virtual Private Cloud (VPC). In accordance with the model, the presentation tier uses an Application Load Balancer running on a public subnet while the application tier uses EC2 instances running on the private application subnet. Finally, the database tier utilizes Amazon RDS, residing in the dedicated private database subnet. The fact that this tier cannot be accessed directly from the Internet can be seen as a built-in protection against the network flatness exploited during the attack at Alamin Clinic.

The proposed design differs from the current design in that it makes use of a three-tiered structure compared to a single-server flat approach used now (Section 2.2.4). The new design includes three tiers that will each have their own separate security group policy, as well as having the database without an outbound Internet route(Gaber *et al.*, 2025).

2.5.3 AWS and the Shared Responsibility Model

The Amazon Web Services runs on a shared responsibility model where there are clearly defined boundaries about who is accountable for what part of the security. It states:

> "AWS manages security of the cloud, you are responsible for security in the cloud."

This means that AWS will take care of the physical security of data centers, the hypervisor level, and the managed services itself. While the customer—the clinic in the case is accountable for OS management, Network Security Groups rules, IAM policies, application security, and data encryption.

It is crucial in the design process for the new system since moving away from traditional hosting to the cloud doesn't relieve the clinic of any security concerns; instead, it shifts them around. It changes the list of those tasks that have been

historically difficult for Alamin Clinic to manage manually to the set that can be easily automated using Terraform, KMS, and IAM policies(Mendoza and Reyes, 2023).

2.5.4 Infrastructure as Code (Terraform)

IaC refers to the use of machine-readable configuration files in lieu of manual procedures to describe and provision computing infrastructure. According to Paidy and Chaganti (2024), IaC significantly decreases configuration drift, making the infrastructure failure recoverability fast and reliable in multi-region AWS deployments—a finding that is directly applicable to the proposed recovery solution.

The chosen IaC tool is HashiCorp's Terraform, which supports a declarative configuration approach that allows the representation of the whole topology of the VPC network, EC2 instance settings, RDS parameter groups, IAM policies, and security group rules in version-controlled code. In effect, the entire environment could be rebuilt from scratch in minutes—an ability that allows turning the Recovery Time Objective into a quantifiable metric.

The discovery directly correlates with the most important insight gained during the stakeholder interview when the IT Administrator was asked about how long it takes to recover from the ransomware attack, and the answer was a period of five days. The recommended RTO target of the system should be less than fifteen minutes, which is possible only due to the configuration being scripted in Terraform and deployable to a clean AWS environment with one command.

Checkov is a static code analyzer tool for IaC that is included in the CI/CD pipeline to scan Terraform scripts before the infrastructure is put to production to avoid common infrastructures security pitfalls, such as excessive permissions on security groups or unprotected storage volumes(Verdet *et al.*, 2023; Espinha Gasiba *et al.*, 2021).

2.5.5 DevSecOps and Shift-Left Security

DevSecOps refers to the process of incorporating security testing and verification during the software development and deployment process, as opposed to

mandating security as a gate for the software development and deployment process after development(Valdés-Rodríguez *et al.*, 2023). "Shift-left" involves moving security tests from their current position closer to the development stage, where issues are detected and fixed at the least cost, even before they become part of the production code base (Paidy and Chaganti, 2024).

In the current deployment process, there is no security gate at any stage. This process involves pushing the code to the production server using FTP. In the new system, there is a CI/CD pipeline in GitHub Actions that carries out security scans in three types on each commit:

The proposed CI/CD pipeline integrates three specialized security scanning tools to ensure a "shift-left" security posture:

1. **Trivy:** Scans Docker container images for known Common Vulnerabilities and Exposures (CVEs) before any image is pushed to the container registry.
2. **SonarQube:** Performs Static Application Security Testing (SAST) by analyzing the application source code for security anti-patterns, injection vulnerabilities, and code quality issues.
3. **Checkov:** Analyzes Terraform configuration files for infrastructure-level security misconfigurations, ensuring that the defined AWS resources adhere to best practices

Any pipeline breach during a critical discovery ensures that the software will not be released, guaranteeing that no software containing a critical flaw is ever deployed into the production environment. The move from no security gate to an automated blocking gate in three stages will solve the reactive security stance described in Section 2.2.2(Rajapakse *et al.*, 2022; Akbar *et al.*, 2022).

2.5.6 Identity and Access Management and Role-Based Access Control

IAM refers to the policies, procedures, and technologies that control the access to certain resources by a particular user within the system. IAM on AWS provides

granular and rule-based control over access to infrastructure components, thereby complementing the application-based RBAC mechanism.

Singh and Chatterjee (2015) proved that for effective security management in the cloud computing environment, there is a need for security enforcement across various tiers, ranging from the application login page to network and resource levels. This paper's proposed framework follows this approach, as it combines AWS IAM policies, which restrict the API calls available to each application module, with application-level RBAC, which restricts the data that each authorized user can read/write, and PostgreSQL Row Level Security, which enforces isolation of data at the storage level, irrespective of the application level.

Three IAM roles will be created for the intended system according to the clinic's three user groups – Doctor, Admin, and Patient. Specifically, the Doctor role will have read/write access to their patients' information, the Admin role will be allowed to read/write information concerning scheduling and registrations, and the Patient role will only have read access to their own information. Each IAM role will have minimum privileges sufficient for their intended purpose(Butt *et al.*, 2023).

PostgreSQL row-level security as an additional control measure will address the issue raised in Section 2.3 regarding inadequate controls on the access aspect of the current system design since the application-based role access controls offer zero security against direct access to the underlying database, which is what ransomware attackers exploit(Cobrado *et al.*, 2024).

2.5.7 HIPAA Compliance

The Health Insurance Portability and Accountability Act (HIPAA) is the main regulatory framework used in securing and protecting patient health information within the United States and is considered a gold-standard benchmark for healthcare data security across the world. HIPAA Security Rule outlines the technical requirements of access control, audit logging, integrity of data, and encryption during data transmission, all of which relate directly to the IAM, CloudTrail, KMS, and TLS of the proposed system(Abbasi and Smith, 2024).

AWS Security Hub is an AWS-managed service designed to provide continuous posture assessment of the AWS environment in relation to HIPAA compliance and provides detailed assessments where there are discrepancies between the configuration and the standards. The suggested architecture uses AWS Security Hub to measure and assess HIPAA posture compliance.

The current framework does not meet the criteria for any of the technical safeguards prescribed by HIPAA, which includes failure to provide proper access control at the infrastructure level, failure to maintain an audit log of data access activities, absence of data integrity through encryption, and lack of reliable transmission security measures. The recommended framework caters to all four areas, thus positioning HIPAA readiness as the benchmark for the assessment process.

2.6 Chapter Summary

This chapter provides the theoretical and the comparative groundwork for the design of the proposed Secure Cloud-Based Patient Data Management System.

Through the analysis of the Alamin Clinic case study based on structured interviews with stakeholders, four problems with the current on-premises system have been identified: flat network architecture, poor access controls, lack of encryption of data, and lack of system resilience. with the existing of these four deficiencies together the ransomware attack took place and led to five days of downtime and loss of data. Through the use of the 5W 1H approach. these problems have been categorized along six dimensions.

Workflow analysis suggests that the existing manual, unsecured process may be replaced with the suggested secure automated workflow, which includes: Flat architecture deployed using three layered VPC; FTP deployment is now replaced with CI/CD pipeline for DevSecOps; Roles is set on a basis of trust using JWT and row level security enforcement; redeployment using Terraform in fifteen minutes; and Audit is not conducted due to usage of CloudTrail and logs.

Literature review evidence is provided here to back up both the academic and business community consensus on all chosen core technologies: three-tier VPC design to ensure network isolation, use of Infrastructure as Code through Terraform to ensure that infrastructure is easily repeatable and quickly restorable, use of DevSecOps pipeline to assist in vulnerability detection earlier in the process, multiple access control through IAM/RBAC and PostgreSQL's row-level security, and HIPAA compliance through AWS Security Hub. Chapter 3 provides definition of development methodology.

CHAPTER 3

SYSTEM DEVELOPMENT METHODOLOGY

3.1 Introduction

This chapter details the development methodology adopted for the Secure Cloud-Based Patient Data Management System and justifies the selection of that methodology in the context of the project's requirements. The design and deployment of a secure cloud infrastructure shows a distinct set of procedural challenges: the system must be developed in iterations to allow the security requirements to be validated on each stage, the pipeline should integrate security tools that execute after every code change, and the infrastructure will follow an incremental build, test, and hardening without requiring a complete rebuild between phases.

Section 3.2 presents the chosen methodology, An Agile development integrated with DevSecOps practices and justifies this choice against alternative methodology. Section 3.3 describes the phases of the methodology as applied to the proposed project with each phase and its deliverables. Section 3.4 provides a technical walkthrough of each tool and technology used in the system. Section 3.5 presents the complete system requirement analysis, covering both functional and non-functional requirements. Section 3.6 summarises the chapter.

3.2 Methodology Choice and Justification

3.2.1 Overview of Candidate Methodologies

Three development methodologies were considered for this project: the Waterfall model, Scrum (Agile), and a DevSecOps integrated Agile approach.

Waterfall it follows a strict sequential approach, the requirements, design, implementation, testing, and deployment, in which each phase must be fully completed before the next begins. This model is appropriate for projects with stable and well understood requirements. However, it is not suited for a cloud security projects

where requirements specially security requirements can emerge from the findings of one phase and must inform the design of the next. In a Waterfall model, a security misconfiguration discovered during the testing phase would require revisiting the design phase, which will need a significant rework.

Scrum is an Agile methodology where development is divided into sprints of predefined duration, creating an increment at the end of each sprint that can be potentially shipped. While the iterative approach of Scrum is well suited for system development in general, the methodology does not define when, and under what conditions, security testing takes place during the sprint. Absent from the process of integrating security practices within sprints, Scrum runs the risk of approaching security separately from functionality and applying security only after the development cycle.

Agile with DevSecOps integrationexpands on Scrum by introducing security testing, scanning for vulnerabilities, and compliance into the sprint cycle and CI/CD pipeline. In other words, instead of performing security tests at some phase following the implementation, they are integrated directly into the process itself such that the product increments have inherent security properties. DevSecOps integration into the Agile cycle is referred to as a natural extension of Agile to the sphere of security, where, just as Agile blurred the lines between the phases of requirements gathering and implementation, DevSecOps does the same to development and security. This principle has been formulated early on in DevSecOps research (Bass *et al.*, 2015), confirmed by more recent research on resilient cloud deployments Paidy and Chaganti (2024). Paidy and Chaganti (2024)

3.2.2 Justification for Agile with DevSecOps

The Agile with DevSecOps methodology is selected for this project on the basis of four aligned criteria.

Iterative security validation. The system's security architecture, VPC network topology, IAM policy structure, encryption configuration, and CI/CD pipeline should be tested at every step of building process. It is not sufficient to test it once.

Due to the agile nature of sprints, every piece of infrastructure can be constructed, analyzed, and verified before another piece is relying on its work. In this way, security debts will not accumulate during different stages of development.

Shift-left security integration. The early execution of security tests in DevSecOps by ensuring that security is integrated as early as possible in the software development process applies to this particular case. The integration of Trivy, SonarQube, and Checkov into the GitHub Actions CI/CD pipeline ensures that every commit runs a security test automatically, hence identifying vulnerabilities during commit and not deployment when it becomes expensive to fix.

Infrastructure as Code compatibility. Since infrastructure is declared in Terraform, it fits into Agile development methodology very well. This means that at each sprint, developers will have a version-controlled configuration for their environment. It would be possible to rebuild the entire environment starting with any version of the code and even roll it back to an earlier version if necessary.

Measurable compliance. The HIPAA compliance goal demands that security postures are constantly measured rather than being evaluated only once at the end of the development process. With AWS Security Hub, such constant assessments are possible and can even be measured sprint after sprint as more infrastructure elements come into the picture.

Table 3.1 summarises the comparison of the three candidate methodologies against the project's key requirements.

Table 3.1 Methodology Comparison

Criteria	Waterfall	Scrum (Agile)	Agile + DevSecOps
Iterative development	×	✓	✓
Security integrated at each stage	×	×	✓
Supports IaC incremental build	Partial	✓	✓
Automated pipeline compatibility	×	Partial	✓
Continuous compliance measurement	×	×	✓
Suited for changing security requirements	×	✓	✓

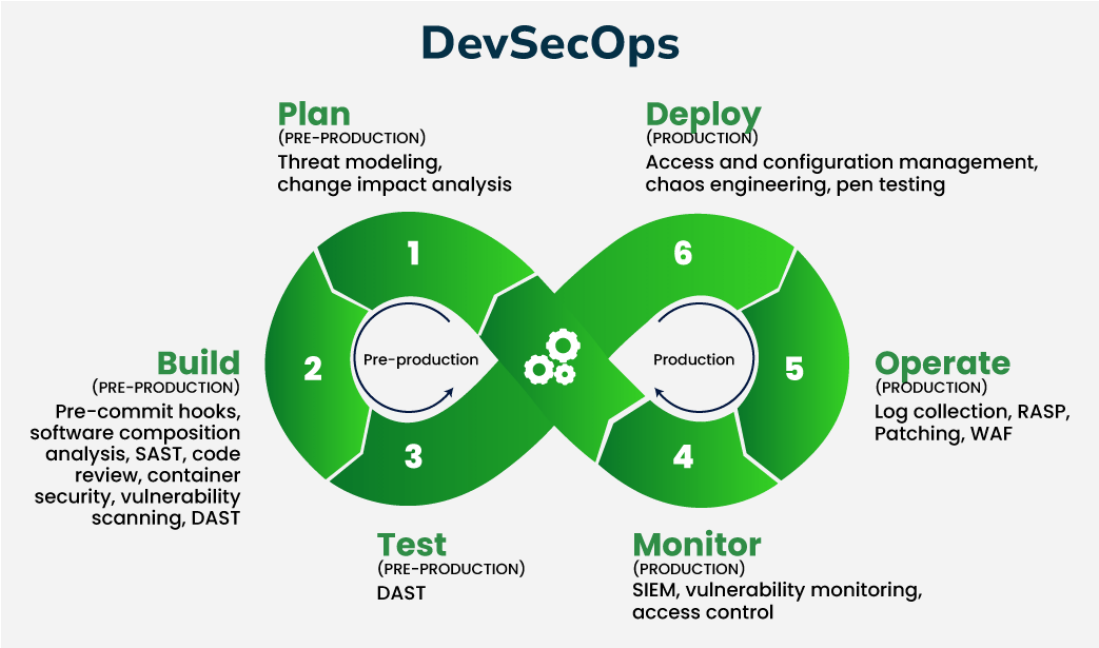


Figure 3.1 Agile + DevSecOps Sprint Cycle with Integrated Security Gates

3.3 Phases of the Chosen Methodology

The project is organized into five sprints. Each sprint has a defined scope, a set of deliverables, and a security gate that must pass before the sprint is considered complete.

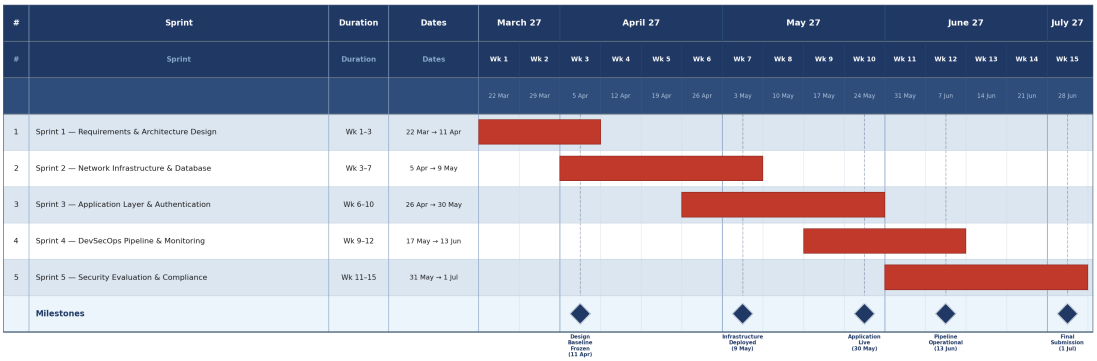


Figure 3.2 Project Schedule: Five-Sprint Development Plan Agile Timeline)

3.3.1 Sprint 1 Requirements and Architecture Design

Scope: List and document all functional and non-functional system requirements. Produce the complete system architecture design, including the three-tier VPC network diagram, IAM policy structure, and DevSecOps pipeline specification.

Deliverables: System requirements document (Chapter 3.5), VPC network architecture diagram, IAM policy matrix, CI/CD pipeline design.

Security gate: Architecture review against HIPAA Security Rule technical safeguard requirements. All identified control gaps must be documented and addressed in the design before Sprint 2 proceeds.

3.3.2 Sprint 2 Network Infrastructure and Database Layer

Scope: Provision the AWS VPC using Terraform, including the public subnet (Application Load Balancer), private application subnet (EC2), and isolated database subnet (RDS). Configure Security Groups, Network ACLs, NAT Gateway, and Internet Gateway. Deploy and configure the Amazon RDS database instance with KMS encryption at rest.

Deliverables: Terraform configuration files for VPC, subnets, routing tables, security groups, and RDS instance. Checkov scan report for all IaC configurations.

Security gate: Checkov scan must report zero critical misconfigurations. RDS instance must be confirmed as unreachable from the public internet. Encryption at rest must be verified in the AWS Console.

3.3.3 Sprint 3 Application Layer and Authentication

Scope: Deploy the Docker-containerised application that consist of React frontend and Node.js/Express backend to EC2 instances in the private application subnet. Implement the RBAC authentication system with three roles (Doctor, Admin, Patient). Configure the Application Load Balancer with HTTPS termination and TLS certificate.

Deliverables: Dockerfiles for frontend and backend, application deployment Terraform modules, IAM role definitions, Trivy scan report for container images.

Security gate:Trivy scan must report zero critical CVEs in deployed container images. TLS termination must be verified at the ALB. Application-level RBAC must be confirmed to enforce role boundaries through test cases covering each user role.

3.3.4 Sprint 4 DevSecOps Pipeline and Monitoring

Scope: Build and configure the complete GitHub Actions CI/CD pipeline, integrating Trivy, SonarQube, and Checkov as automated pipeline stages. Configure Amazon CloudWatch log groups and metric alarms. Configure AWS CloudTrail for audit logging of all API calls and data access events.

Deliverables: GitHub Actions workflow YAML files, SonarQube configuration, CloudWatch dashboard configuration, CloudTrail trail configuration.

Security gate:Pipeline must demonstrate that a commit containing a deliberate critical vulnerability (test CVE injection) is blocked before reaching the deployment stage. CloudTrail must be confirmed to capture patient data access events.

3.3.5 Sprint 5 Security Evaluation and Compliance Testing

Scope: Execute the full security evaluation framework: automated vulnerability scan reports from Trivy, SonarQube, and Checkov; black-box and white-box penetration testing of the application; Recovery Time Objective stress test simulating a ransomware wipe-and-redeploy scenario; and HIPAA compliance posture assessment via AWS Security Hub.

Deliverables: Security scan reports, penetration testing results, RTO test log with measured recovery time, AWS Security Hub HIPAA findings report.

Security gate: RTO must be measured and documented. Security Hub HIPAA posture score must be recorded as the primary compliance metric. All critical Security Hub findings must be remediated before the sprint is closed.

3.4 Technology Used Description

This section provides a technical description of each tool and service used in the proposed system, organized by functional category.

3.4.1 Cloud Infrastructure: Amazon Web Services (AWS)

Amazon Web Services provides the cloud infrastructure platform for the proposed system. The following AWS services are employed:

Amazon Virtual Private Cloud (VPC) enables the creation of logical network isolation through the AWS cloud. The recommended design involves configuring one VPC comprising three subnets; namely, public subnet, private application subnet, and private database subnet. This setup will involve having these subnets across multiple Availability Zones. The VPC becomes the key tool in achieving network segmentation architecture at Alamin Clinic.

Amazon EC2 (Elastic Compute Cloud) provides the virtual server instances on which the application backend runs. EC2 instances are deployed within the private application subnet, accessible only through the Application Load Balancer, and are not directly reachable from the internet.

Amazon RDS (Relational Database Service) The managed relational database is provided by RDS. The service is installed in the private subnet without internet access. Data at rest encryption is done by KMS, and backups have been set up for point-in-time recovery. The system utilizes Amazon RDS for PostgreSQL which is running in the private subnet with KMS at rest encryption.

Application Load Balancer (ALB) distributes incoming HTTPS traffic across EC2 instances. The ALB terminates TLS, ensuring that all traffic entering the application layer is encrypted in transit. It is the only entry point to the application from the public internet.

AWS Key Management Service (KMS) provides managed cryptographic keys for encrypting the RDS database and any sensitive data stored in Amazon S3. AES-256 encryption is applied at rest.

Amazon CloudWatch provides monitoring and alerting for the deployed infrastructure. Log groups capture application logs and system metrics; alarms are configured to trigger notifications on anomalous patterns such as repeated authentication failures or unusual API call volumes.

AWS CloudTrail records every AWS API call made within the account, creating an immutable audit log of all infrastructure changes and data access events. This satisfies the HIPAA requirement for audit logging and provides forensic capability in the event of a security incident.

AWS Security Hub aggregates security findings from multiple AWS services and evaluates the account's security posture against the HIPAA Security Standard. Security Hub is used as the primary compliance measurement tool for this project.

3.4.2 Infrastructure as Code: Terraform

Terraform, developed by HashiCorp, is the Infrastructure as Code tool used to define, provision, and manage all AWS resources in this project. Terraform uses a declarative configuration language (HCL — HashiCorp Configuration Language) in

which the desired end state of the infrastructure is specified, and Terraform determines the sequence of API calls required to achieve that state.

The entire system which include VPC configuration, subnet routing, security group rules, EC2 instance specifications, RDS parameter groups, IAM policies, and ALB listener rules is expressed as version-controlled Terraform configuration files. This approach provides three security-relevant capabilities: it creates a verifiable record of every infrastructure change through git history; it enables complete environment destruction and redeployment from a clean state within minutes (directly addressing the ransomware recovery objective); and it allows Checkov to statically analyse the configuration for security misconfigurations before any resource is provisioned.

3.4.3 Containerisation: Docker

Docker is used to package the application frontend and backend as portable container images. Containerisation ensures that the application runtime environment is consistent across development, testing, and production stages, eliminating configuration drift between environments. Docker images are built during the CI/CD pipeline and scanned by Trivy before being pushed to the container registry.

3.4.4 CI/CD Pipeline: GitHub Actions

GitHub Actions is the CI/CD platform used to automate the build, scan, and deployment pipeline. A workflow defined in YAML is triggered on every push to the repository's main branch. The pipeline executes the following stages in sequence:

(a) Code checkout and dependency installation

(b) SonarQube SAST scan of application source code

(c) Trivy container image vulnerability scan

(d) Checkov IaC misconfiguration scan of Terraform files

(e) Terraform plan and apply (deployment) only if all scan stages pass

A failure at any scan stage with a critical or high-severity finding terminates the pipeline and blocks deployment. This enforces the shift-left principle: no code with a known critical vulnerability can reach the production environment.

3.4.5 Security Scanning Tools

Trivy (Aqua Security) is an open-source vulnerability scanner for container images, file systems, and Git repositories. In this project, Trivy scans Docker images against the National Vulnerability Database (NVD) to identify known CVEs in base images and application dependencies before the image is pushed to the registry.

SonarQube is a Static Application Security Testing (SAST) platform that analyses source code for security vulnerabilities, code quality issues, and injection risks. SonarQube integration in the pipeline ensures that insecure coding patterns such as SQL injection vulnerabilities, hardcoded credentials, or improper input validation are flagged before the code is deployed.

Checkov is a static analysis tool for Infrastructure as Code that evaluates Terraform, CloudFormation, and Kubernetes configurations against a library of security and compliance policies. In this project, Checkov enforces policies such as: RDS instances must have encryption at rest enabled; S3 buckets must not be publicly accessible; Security Groups must not permit unrestricted inbound SSH access; and CloudTrail must be enabled.

3.4.6 Application Stack

React is the JavaScript framework used for the system's web frontend, providing the patient portal, appointment scheduling interface, and administrative dashboards. The React application is served as a static bundle through the Application Load Balancer.

Node.js with Express is the JavaScript runtime and web framework used for the application backend, providing the RESTful API layer that handles authentication, role enforcement, and data access logic. Node.js/Express is selected over alternatives for three reasons: the development team has direct prior experience with this

stack, eliminating framework learning overhead during the project; the same JWT authentication, bcrypt password hashing, and cookie-based session patterns validated in prior development work carry over directly; and a single language (JavaScript) across both the React frontend and the Express backend reduces context switching and simplifies the Docker containerisation setup.

PostgreSQL is the relational database management system deployed on Amazon RDS. PostgreSQL was selected for its robust support for row-level security policies, which complement the RBAC model by allowing database-level access restrictions to be defined per user role.

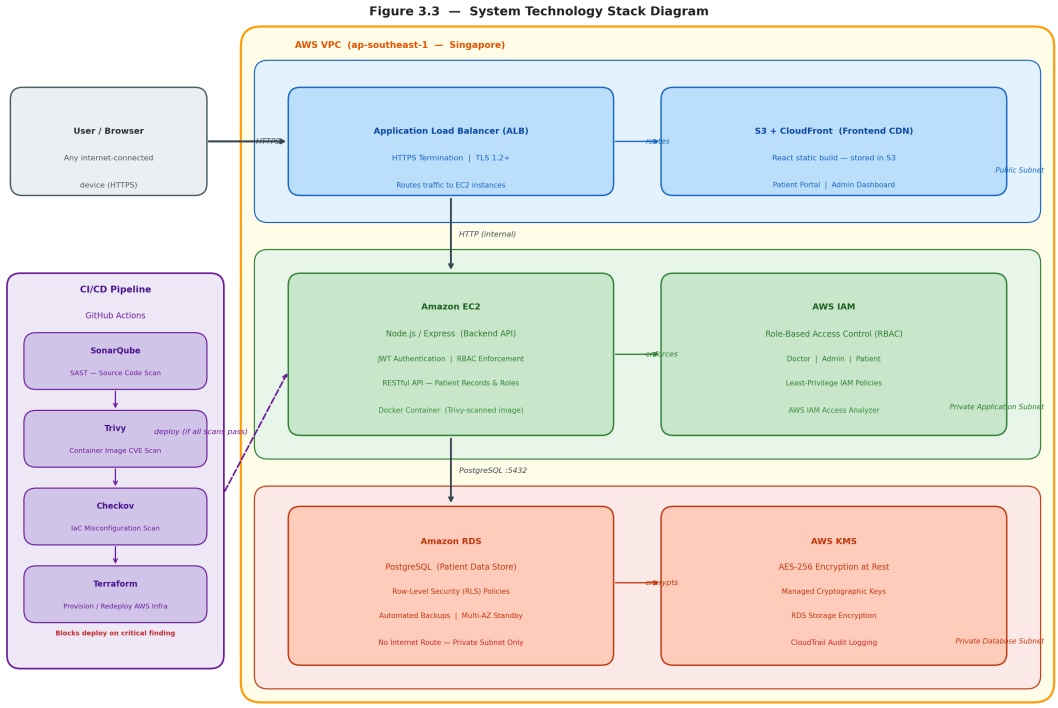


Figure 3.3 System Technology Stack Diagram

3.5 System Requirement Analysis

System requirements are divided into two categories: functional requirements, which define what the system must do, and non-functional requirements, which define the quality constraints under which it must operate. Both categories are derived directly from the deficiencies identified in the Alamin Clinic case study (Chapter 2) and from the HIPAA Security Rule technical safeguard requirements.

3.5.1 Functional Requirements

Functional requirements specify the behaviours and operations the system must support. Table 3.2 lists the functional requirements for the proposed system.

Table 3.2 Functional Requirements

ID	Requirement	User Role	Priority
FR-01	The system shall allow new patients to be registered with a unique identifier, personal details, and assigned doctor.	Admin	High
FR-02	The system shall allow authenticated doctors to create, read, and update medical records for patients assigned to their care.	Doctor	High
FR-03	The system shall allow authenticated doctors to read the medical history of their assigned patients.	Doctor	High
FR-04	The system shall allow authenticated administrators to create, update, and cancel patient appointments.	Admin	High
FR-05	The system shall allow authenticated patients to view their own upcoming and past appointments.	Patient	High
FR-06	The system shall allow authenticated patients to view their own medical records in read-only mode.	Patient	High
FR-07	The system shall authenticate all users with a unique username and password before granting access to any system function.	All	High
FR-08	The system shall enforce role-based access control, ensuring that each user role can only access the data and functions authorised for that role.	All	High

ID	Requirement	User Role	Priority
FR-09	The system shall log all patient data access events, including the user identity, timestamp, and action performed.	System	High
FR-10	The system shall allow administrators to create, deactivate, and reassign user accounts.	Admin	Medium
FR-11	The system shall transmit all data between the client browser and the server over HTTPS.	System	High
FR-12	The system shall store all patient records in an encrypted database with no direct internet access path.	System	High

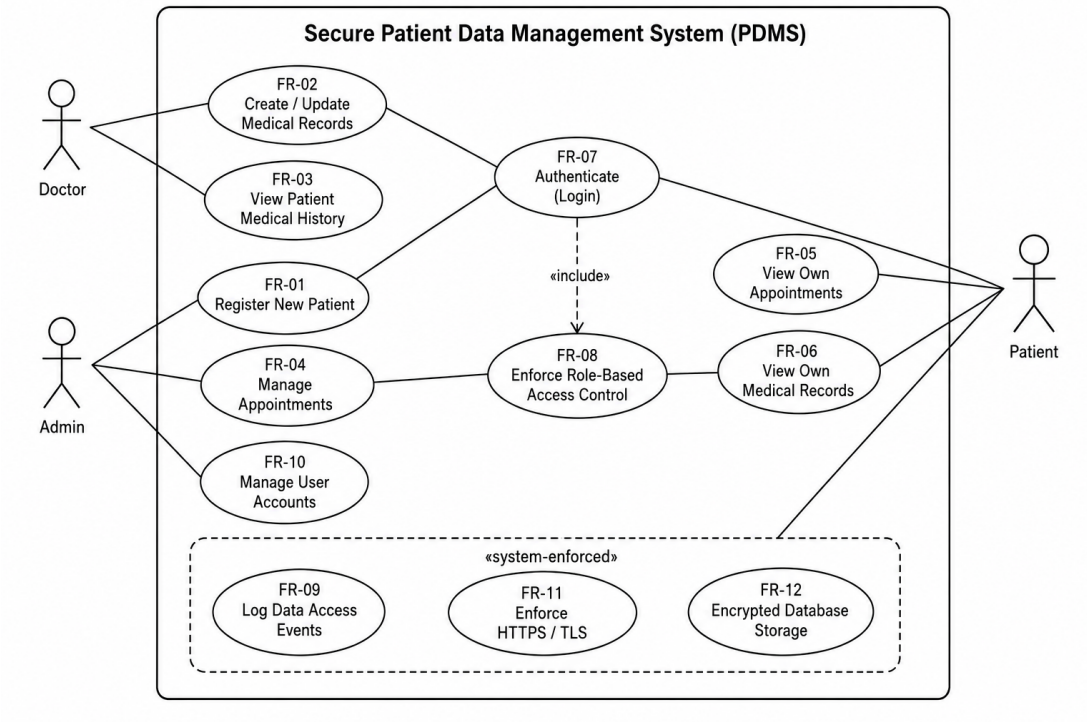


Figure 3.4 requirements traceability diagram

3.5.2 Non-Functional Requirements

Non-functional requirements define the performance, security, reliability, and compliance constraints that the system must satisfy. Table 3.3 lists the non-functional requirements.

Table 3.3 Non-Functional Requirements

ID	Category	Requirement	Metric / Verification Method
NFR-01	Security	All data stored in the RDS database shall be encrypted at rest using AES-256 via AWS KMS.	AWS Console, RDS encryption status
NFR-02	Security	All data in transit between the client and ALB shall be encrypted using TLS 1.2 or higher.	SSL Labs scan, ALB listener configuration
NFR-03	Security	All IAM roles shall be configured with least-privilege policies, granting only the permissions required for the role's function.	IAM policy review, AWS IAM Access Analyzer
NFR-04	Security	The CI/CD pipeline shall block deployment on any critical or high-severity finding from Trivy, SonarQube, or Checkov.	GitHub Actions pipeline log
NFR-05	Availability	The system shall maintain 99.9% uptime through multi-AZ EC2 and RDS deployment.	CloudWatch availability metric
NFR-06	Recovery	The system shall be fully redeployable from a clean Terraform state within 15 minutes of a complete infrastructure wipe.	RTO stress test, measured recovery time
NFR-07	Compliance	The system shall achieve and maintain a passing HIPAA posture score as measured by AWS Security Hub.	Security Hub HIPAA standard findings report
NFR-08	Auditability	All AWS API calls and patient data access events shall be logged in CloudTrail with a minimum retention period of 90 days.	CloudTrail configuration, S3 log bucket

ID	Category	Requirement	Metric / Verification Method
NFR-09	Performance	The system shall respond to authenticated API requests within 3 seconds under normal load (up to 50 concurrent users).	Load test results
NFR-10	Scalability	The application tier shall support horizontal scaling through EC2 Auto Scaling to accommodate increased patient load.	Auto Scaling group configuration
NFR-11	Maintainability	All infrastructure shall be defined as version-controlled Terraform code, with no manually provisioned resources in the production environment.	Terraform state file audit

3.5.3 Minimum System Requirements

This section specifies the minimum hardware and software requirements for each category of user to access and operate the system, and the minimum server-side specifications required to deploy the system on AWS.

Table 3.4 Minimum Client-Side Requirements (End User)

Requirement	**Minimum Specification**	**Recommended**
Device	Any internet connected device (PC, laptop, tablet, or smartphone)	Desktop or laptop
Processor	1 GHz single-core	2 GHz dual core or higher
RAM	1 GB	4 GB or higher
Internet Connection	1 Mbps stable broadband	10 Mbps or higher
Web Browser	Google Chrome 90+, Mozilla Firefox 88+, Microsoft Edge 90+, Safari 14+ (JavaScript must be enabled)	Latest version of Chrome or Firefox
Screen Resolution	1280 $\times$ 720 pixels	1920 $\times$ 1080 pixels
Operating System	Windows 7 or later, macOS 10.14 or later, iOS 12 or later, Android 8.0 or later	Windows 10+, macOS 12+

No software installation is required on the client device. The system is delivered as a browser-based web application accessible via HTTPS. All processing occurs on the server side; the client device is only required to render the web interface and transmit user input.

Table 3.5 Minimum Server-Side Requirements (AWS Deployment)

Component	**AWS Service**	**Minimum Instance/Tier**	**Purpose**
Application Server	Amazon EC2	t3.small (2 vCPU, 2 GB RAM)	Node.js/Express backend
Database Server	Amazon RDS	db.t3.micro (2 vCPU, 1 GB RAM)	PostgreSQL patient data store
Load Balancer	Application Load Balancer	Standard (1 LCU)	HTTPS traffic distribution
Storage	Amazon EBS	20 GB GP3 SSD	EC2 root volume
Database Storage	Amazon RDS Storage	20 GB GP2 SSD	Patient records and audit log
Network	AWS VPC	/16 CIDR, 6 subnets across 2 AZs	Network isolation and segmentation

These specifications define the minimum configuration required to run the system in a pilot deployment supporting up to 50 concurrent users. Production scale-up is achieved through EC2 Auto Scaling and RDS instance class upgrades with no change to the application code or network architecture.

3.6 Chapter Summary

This chapter defined the Agile with DevSecOps methodology adopted for the project and justified its selection over Waterfall and standard Scrum approaches on the basis of four criteria: iterative security validation, shift-left pipeline integration, Infrastructure as Code compatibility, and measurable compliance tracking.

The project is structured into five sprints. Sprint 1 covers requirements and architecture design. Sprints 2 through 5 cover network infrastructure, application layer, pipeline integration, and security evaluation respectively. Each sprint is bounded by a security gate that must be cleared before the next sprint proceeds.

The technology stack was described across six categories: AWS cloud services (VPC, EC2, RDS, ALB, KMS, CloudWatch, CloudTrail, Security Hub), Terraform IaC, Docker containerisation, GitHub Actions CI/CD, security scanning tools (Trivy, SonarQube, Checkov), and the application stack (React, Node.js/Express, PostgreSQL). Each technology selection was grounded in the security and operational requirements established in the literature review.

The system requirement analysis produced twelve functional requirements and eleven non-functional requirements, each with a defined verification method. The non-functional requirements establish the measurable security, availability, recovery, and compliance targets against which the system will be evaluated in Chapter 5. Chapter 4 proceeds to translate these requirements into the complete system design.

CHAPTER 4

REQUIREMENT ANALYSIS AND DESIGN

4.1 Introduction

This chapter describes the system design that has been developed based on the requirements from Chapter 3. It starts with the use case analysis that defines how users of each role will behave in the system. The next step involves describing the architecture design for the system, which includes such aspects as the network topology within AWS, security control setting, IAM policy design, and DevSecOps pipeline design. Next comes the database schema design that includes PostgreSQL table design and row-level security policies that ensure proper data isolation at the storage level.

The requirement analysis is described by user stories and use cases in Section 4.2. The design of the whole project involving system architecture is provided in Section 4.3. Database design and its schema will be provided in Section 4.4. Interface design for all users is given in Section 4.5. Finally, Section 4.6 provides a summary of the chapter.

4.2 Requirement Analysis

4.2.1 Use Case Overview

The system supports three types of users including: Doctor, Admin, and Patient, each with their own set of allowable operations. Figure 4.1 illustrates the use case diagram that includes all actors along with their allowable operations in the system.

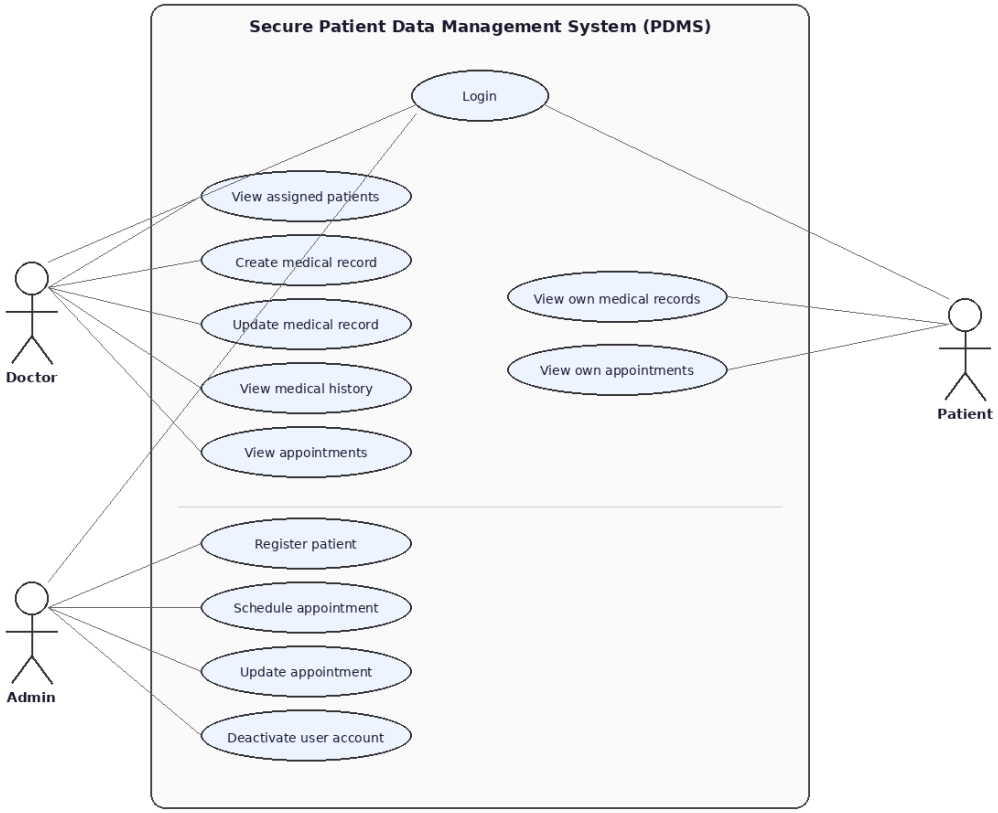


Figure 4.1 UML Use Case Diagram — Secure Patient Data Management System

4.2.2 User Stories

User stories capture the system's functional requirements from the perspective of each actor, following the Agile format: "As a [role], I want to [action], so that [benefit]." Table 4.1 presents all eighteen user stories, organised by module. These stories were derived directly from the stakeholder interviews conducted in Section 2.2.3 and informed the use case design in Section 4.2.3.

Table 4.1 User Stories by Module

ID	Role	User Story	Module
US-01	Doctor / Admin / Patient	As a user, I want to log in with my username and password, so that I can securely access my role-specific dashboard.	Auth

ID	Role	User Story	Module
US-02	All roles	As a logged-in user, I want to log out, so that my session is terminated and my account is protected on shared devices.	Auth
US-03	System	As a user, I want the system to lock my account after three consecutive failed login attempts, so that brute-force attacks are prevented and the admin is alerted.	Auth
US-04	Admin	As an Admin, I want to create user accounts with assigned roles, so that doctors, staff, and patients can access the system with the correct permissions.	Auth
US-05	Admin	As an Admin, I want to deactivate user accounts, so that former staff or inactive patients can no longer access the system.	Auth
US-06	Admin	As an Admin, I want to register new patients and assign them a treating doctor, so that their records can be managed securely from the point of registration.	Patient Mgmt
US-07	Doctor / Admin	As a Doctor or Admin, I want to view a patient's profile, so that I can review their details before providing care or scheduling an appointment.	Patient Mgmt
US-08	Admin	As an Admin, I want to update patient demographic information, so that the system holds accurate and current patient details.	Patient Mgmt
US-09	Admin	As an Admin, I want to assign or reassign a treating doctor to a patient, so that the correct doctor has access to that patient's records.	Patient Mgmt
US-10	Doctor	As a Doctor, I want to create medical records for my assigned patients, so that I can document diagnoses and prescriptions in a secure, auditable system.	Medical Records

ID	Role	User Story	Module
US-11	Doctor / Patient	As a Doctor, I want to view records of my assigned patients; as a Patient, I want to view my own records in read-only mode, so that clinical data is accessible only to authorised parties.	Medical Records
US-12	Doctor	As a Doctor, I want to update a record I created, so that I can correct or supplement clinical documentation after the initial consultation.	Medical Records
US-13	Doctor	As a Doctor, I want to view the complete chronological medical history of my assigned patients, so that I can make informed clinical decisions.	Medical Records
US-14	Admin	As an Admin, I want to schedule appointments linking a patient to a doctor at a specific date and time, so that consultations are organised and conflict-free.	Appointments
US-15	Doctor	As a Doctor, I want to view my appointment schedule, so that I know which patients I will be seeing and can prepare for each consultation.	Appointments
US-16	Patient	As a Patient, I want to view my upcoming appointments in read-only mode, so that I am informed of when and with whom my consultations are scheduled.	Appointments
US-17	Admin	As an Admin, I want to update appointment details, so that changes in scheduling requirements are reflected accurately in the system.	Appointments

ID	Role	User Story	Module
US-18	Admin	As an Admin, I want to cancel appointments, so that unavailable time slots are freed and appointment records remain accurate for audit purposes.	Appointments

4.2.3 Use Case Descriptions

The following tables describe the primary use cases in detail, specifying the actor, preconditions, main flow, and exception handling for each.

4.2.3.1 Use Case UC-01: Doctor Creates Medical Record

Table 4.2 Use Case UC-01: Doctor Creates Medical Record

Field	**Description**
Use Case ID	UC-01
Use Case Name	Create Medical Record
Actor	Doctor
Precondition	Doctor is authenticated. Patient is registered and assigned to this doctor.
Main Flow	(a) Doctor selects patient from assigned patient list. (b) Doctor selects ”New Medical Record”. (c) System displays record creation form. (d) Doctor enters diagnosis, prescription, and clinical notes. (e) Doctor submits the record. (f) System validates input and writes the record to the database with the doctor’s ID and a timestamp. (g) System generates an audit log entry recording the creation event. (h) System confirms successful save to the doctor.
Alternative Flow	If the patient is not assigned to this doctor, the system returns an authorisation error and the record is not created.
Postcondition	A new medical record is stored in the database, associated with the patient and the treating doctor. An audit log entry exists for the event.

4.2.3.2 Use Case UC-02: Admin Schedules Appointment

Table 4.3 Use Case UC-02: Admin Schedules Appointment

Field	**Description**
Use Case ID	UC-02
Use Case Name	Schedule Appointment
Actor	Admin
Precondition	Admin is authenticated. Patient and doctor are both registered in the system.
Main Flow	(a) Admin navigates to the appointment scheduling screen. (b) Admin selects the patient and the assigned doctor. (c) Admin selects date and time slot from available slots. (d) System checks for scheduling conflicts. (e) Admin confirms the booking. (f) System creates the appointment record and associates it with both the patient and doctor. (g) System confirms successful scheduling to the admin.
Alternative Flow	If a scheduling conflict exists, the system notifies the admin and returns to step 3.
Postcondition	An appointment record is stored in the database, visible to the relevant doctor, admin, and patient.

4.2.3.3 Use Case UC-03: Patient Views Own Medical Records

Table 4.4 Use Case UC-03: Patient Views Own Medical Records

Field	**Description**
Use Case ID	UC-03
Use Case Name	View Own Medical Records
Actor	Patient
Precondition	Patient is authenticated.
Main Flow	(a) Patient navigates to the medical records section. (b) System queries the database for records where `patient_id` matches the authenticated user. (c) System returns a list of the patient's records ordered by date. (d) Patient selects a record to view the details.
Alternative Flow	If no records exist, the system displays an empty state message.
Postcondition	Patient views their own records only. No other patient's data is returned at any point.

4.2.3.4 Use Case UC-04: User Authentication

Table 4.5 Use Case UC-04: User Authentication

Field	**Description**
Use Case ID	UC-04
Use Case Name	User Login
Actor	Doctor, Admin, Patient (all roles)
Precondition	User account exists and is active.
Main Flow	(a) User navigates to the login screen. (b) User enters username and password. (c) System validates credentials against the hashed password in the database. (d) On success, system issues a JWT token containing the user's role. (e) All subsequent API requests include the JWT token in the `Authorization` header. (f) System validates the token and enforces role permissions on every request.
Alternative Flow	On three consecutive failed login attempts, the account is temporarily locked and the admin is notified.
Postcondition	User is authenticated and operating under the permissions of their assigned role.

4.2.4 Sequence Diagrams

The sequence diagrams show the dynamic interactions between system components based on the four main use cases explained in section 4.2.3. Each sequence diagram depicts the entire path of requests made from the user's browser to the frontend application using React, the API server using Express, and the database server using PostgreSQL, indicating exactly which API endpoint was called, JWT authentication was done, and finally the audit log written.

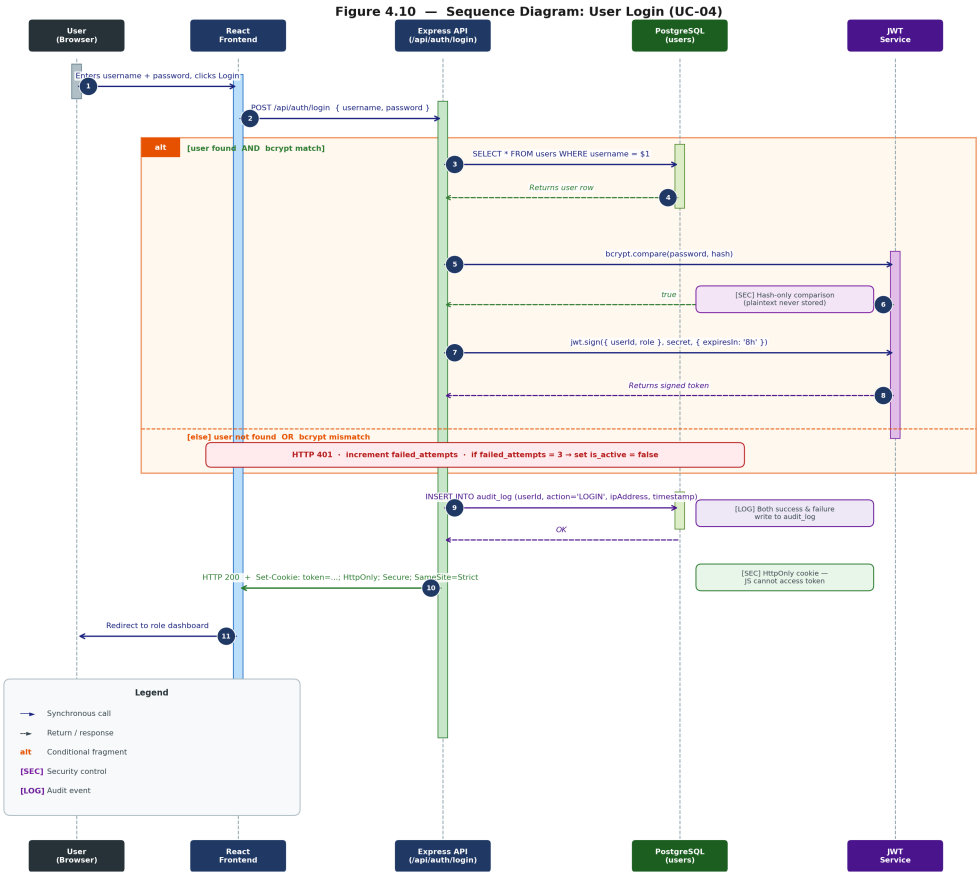


Figure 4.2 Sequence Diagram: User Login (UC-04)

Login is the first security checkpoint for the whole system. Key decisions made in this architecture include: (1) No comparisons of the password are done in clear text; bcrypt.compare() is performed solely on the hashed value; (2) the JWT token is set as an httpOnly cookie so it cannot be accessed via JavaScript; (3) Both successful and unsuccessful authentication writes to the audit log.

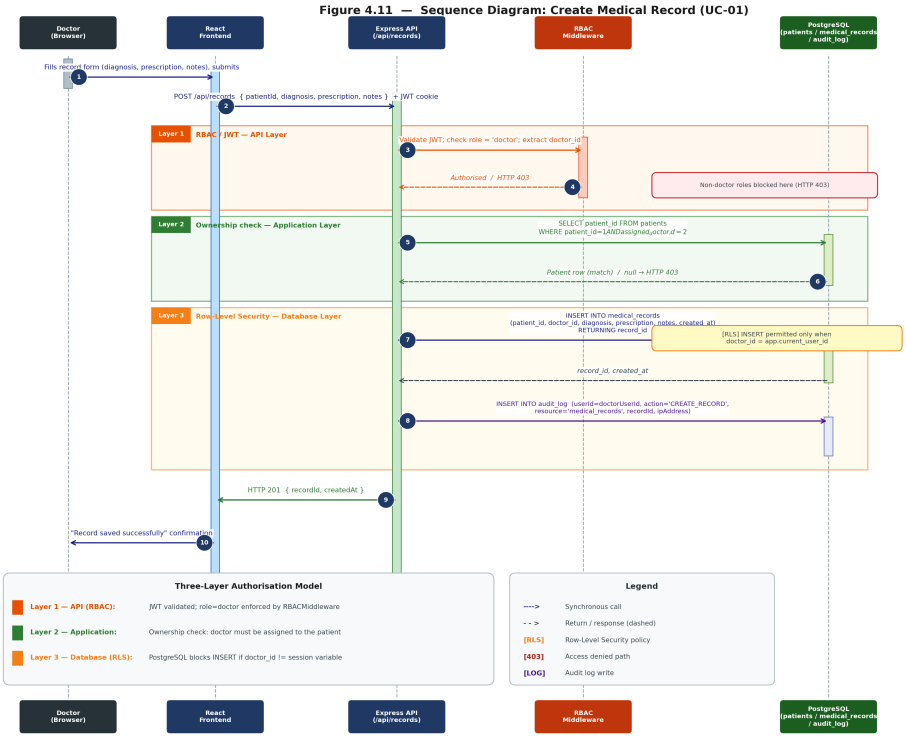


Figure 4.3 Sequence Diagram: Create Medical Record (UC-01)

This figure illustrates the use of the three-layer authorization pattern on the most sensitive write transaction in the application. RBAC Middleware in step 3 prevents all other users except Doctors from accessing the application API. Ownership check at step 5 guarantees that the specific doctor is responsible for the particular patient, which is verified in the application layer. RLS on step 7 denotes enforcement in the database layer.

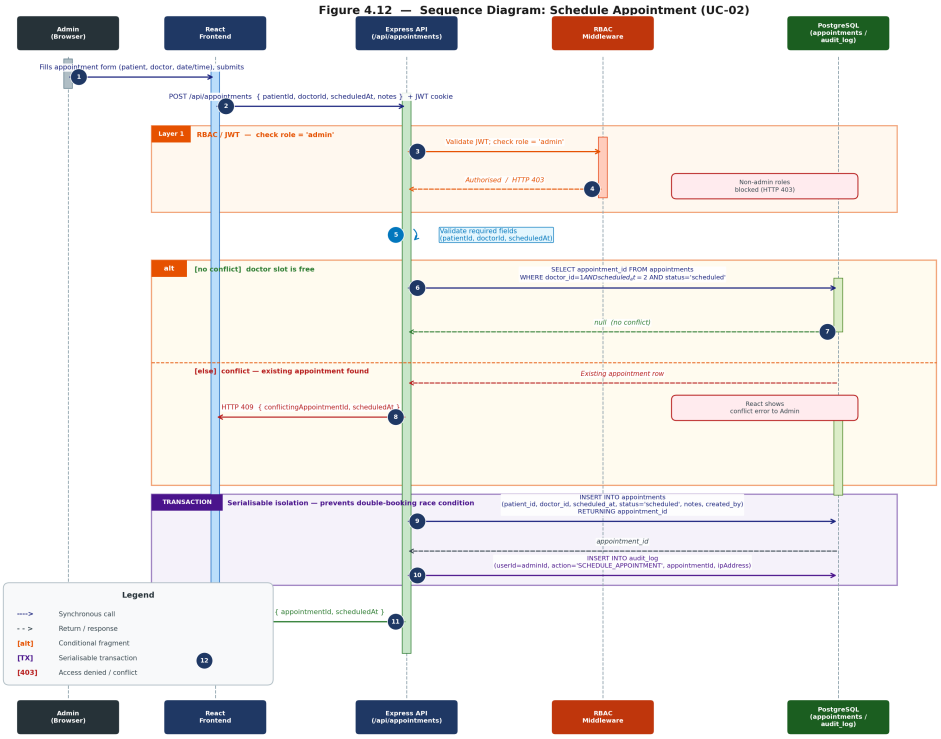


Figure 4.4 Sequence Diagram: Schedule Appointment (UC-02)

The scheduling algorithm brings in the conflict checking mechanism. The database will check whether there exists any appointment for that doctor in the same time slot prior to saving, and return an HTTP 409 response upon finding any conflict. Step 9 and Step 10 are executed in a serializable transaction to avoid any race condition between two admins scheduling the same slot.

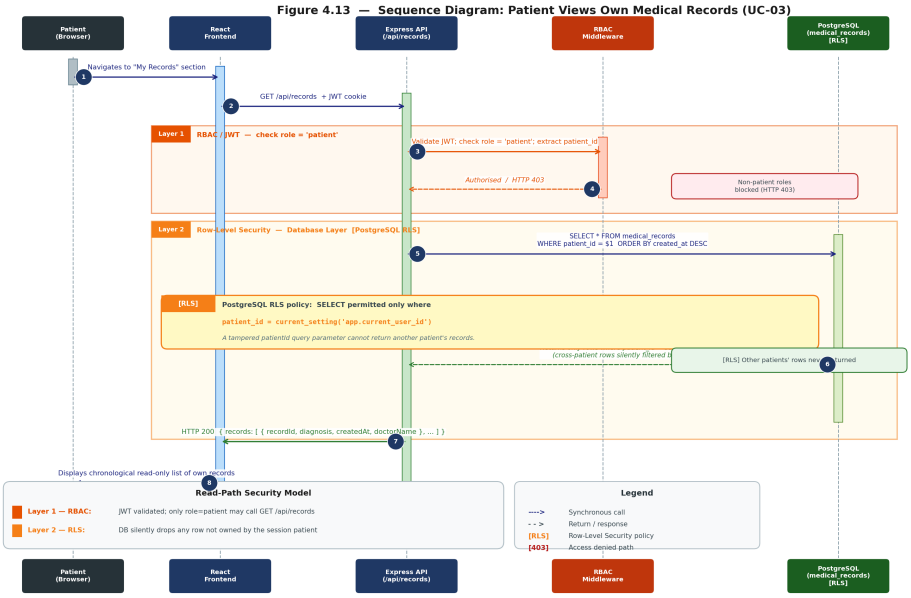


Figure 4.5 Sequence Diagram: Patient Views Own Medical Records (UC-03)

This is the example of how the read-path security approach works for the Patient role. Step 6 here, where RLS is annotated, is the critical step because even when the hacker attempts to manipulate the API request using the altered patient_id, the RLS feature automatically filters out all the records that do not belong to the authenticated session user in the database layer.

4.3 Project Design

4.3.1 System Architecture Overview

The designed architecture for the system uses a three-tiered web application running on the AWS Virtual Private Cloud network. The presentation layer, application layer, and data layer of the system have been designed to be present in separate layers of subnets with their respective access policies. There is no direct connection between the presentation layer and the data layer.

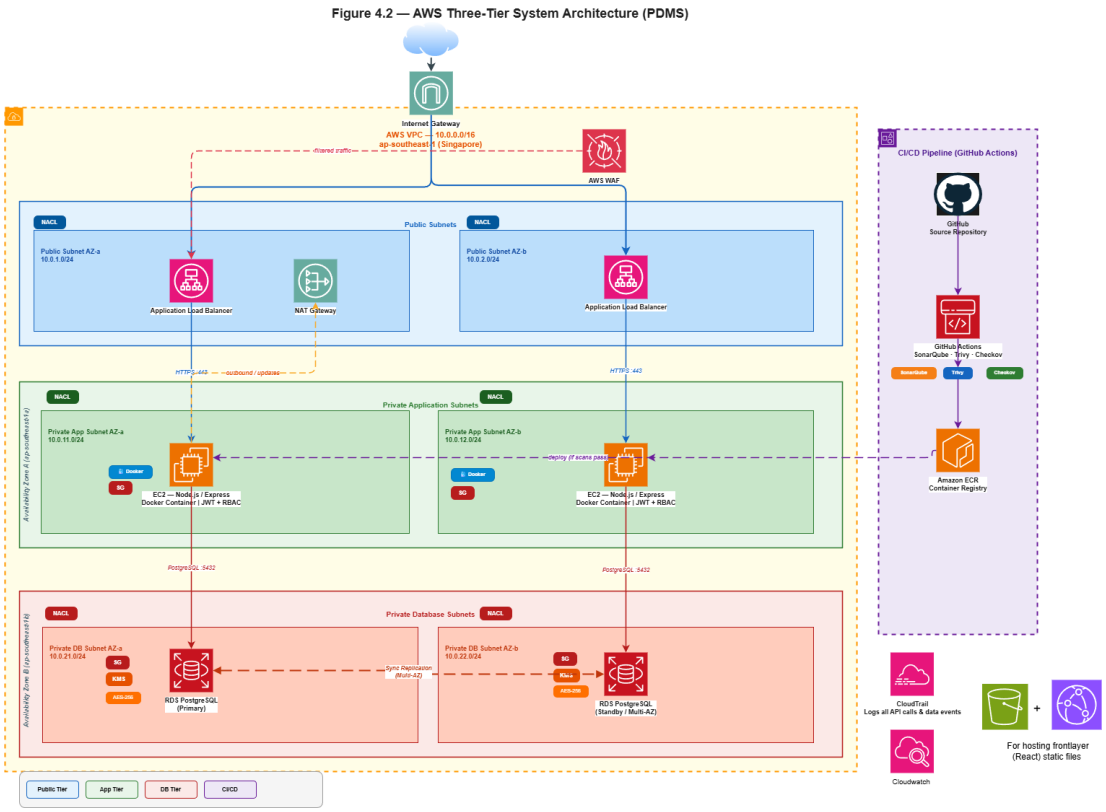


Figure 4.6 AWS Three-Tier System Architecture for the Patient Data Management System (PDMS)

4.3.2 VPC Network Design

The system is deployed within a single AWS VPC with the CIDR block ‘10.0.0.0/16‘. The VPC is divided into six subnets across two Availability Zones with three subnet tiers per AZ — to achieve high availability and fault isolation.

Table 4.6 VPC Subnet Configuration

Subnet	**CIDR**	**Availability Zone**	**Tier**	**Purpose**
public-subnet-a	10.0.1.0/24	ap-southeast-1a	Public	ALB, NAT Gateway
public-subnet-b	10.0.2.0/24	ap-southeast-1b	Public	ALB (multi-AZ)
app-subnet-a	10.0.3.0/24	ap-southeast-1a	Private	EC2 application instances
app-subnet-b	10.0.4.0/24	ap-southeast-1b	Private	EC2 application instances
db-subnet-a	10.0.5.0/24	ap-southeast-1a	Isolated	RDS primary instance
db-subnet-b	10.0.6.0/24	ap-southeast-1b	Isolated	RDS standby instance (Multi-AZ)

Routing configuration. The public subnets are bound to the route table which sends the traffic destined for ‘0.0.0.0/0‘ through the Internet Gateway, thus allowing internet traffic directed to the ALB and outbound internet traffic from NAT. The application subnets are bound to the route table which sends the traffic destined for ‘0.0.0.0/0‘ via the NAT Gateway, thus allowing EC2 instances to establish outbound connections (to AWS API, package repositories, etc.), but not being accessible from the internet itself. Database subnets have no connectivity to the internet at all – their route table consists of only one route which is the local VPC route (‘10.0.0.0/16‘).

4.3.3 Security Group Configuration

Security Groups act as virtual firewalls at the instance level, enforcing allow-list rules for inbound and outbound traffic. Three Security Groups are defined.

Table 4.7 Security Group Rules

Security Group	**Direction**	**Protocol**	**Port**	**Source / Destination**	**Purpose**
alb-sg	Inbound	TCP	443	0.0.0.0/0	HTTPS from internet
alb-sg	Inbound	TCP	80	0.0.0.0/0	HTTP (redirected to 443)
alb-sg	Outbound	TCP	5000	ec2-sg	Forward to application
ec2-sg	Inbound	TCP	5000	alb-sg	Accept only from ALB
ec2-sg	Outbound	TCP	5432	rds-sg	Connect to database
ec2-sg	Outbound	TCP	443	0.0.0.0/0	AWS API / NAT outbound
rds-sg	Inbound	TCP	5432	ec2-sg	Accept only from EC2
rds-sg	Outbound	—	—	None	No outbound permitted

The security feature that is important in this design is the security that the RDS instance will only be accessible through the EC2 Security Group. There is no policy that will allow any other connection – including access directly by developers, the ALB, or any Internet address – to the database port.

4.3.4 Network Access Control List (NACL) Configuration

Network ACLs provide a stateless secondary security boundary at the subnet level, complementing Security Groups. Three NACLs are defined, one per subnet tier.

Table 4.8 NACL Rules Summary

NACL	Applies To	Key Inbound Rules	Key Outbound Rules
public-nacl	Public subnets	Allow TCP 443 from 0.0.0.0/0; Allow TCP 80 from 0.0.0.0/0; Allow ephemeral ports (1024–65535) from 0.0.0.0/0	Allow all to 0.0.0.0/0
app-nacl	App subnets	Allow TCP 5000 from 10.0.1.0/24 and 10.0.2.0/24 (public subnets); Allow ephemeral ports from 10.0.5.0/24 and 10.0.6.0/24 (DB response)	Allow TCP 5432 to 10.0.5.0/24 and 10.0.6.0/24; Allow ephemeral ports to public subnets
db-nacl	DB subnets	Allow TCP 5432 from 10.0.3.0/24 and 10.0.4.0/24 (app subnets) only; Deny all other inbound	Allow ephemeral ports to 10.0.3.0/24 and 10.0.4.0/24 only

NACL is also another defense-in-depth tool since, in case something went wrong with configuring SG rules, any access to port 5432 that was initiated by other than applications subnets would be blocked by NACL as well.

4.3.5 IAM Policy Design

Three IAM roles are assigned, corresponding to the three operational roles of the users within the system. They are assigned minimum necessary privileges to perform their operations on AWS resources based on the concept of least privilege.

Table 4.9 IAM Role Definitions

Role	AWS Permissions	Application Permissions	Scope Restriction
Doctor	`logs:PutLogEvents`	Read/write own assigned patient records; read appointment schedule	Records filtered by `assigned_doctor_id = current_user`
Admin	`logs:PutLogEvents`	Create/update patient registrations; create/update/cancel appointments; deactivate user accounts	No access to medical record content
Patient	`logs:PutLogEvents`	Read own medical records (read-only); read own appointments (read-only)	Records filtered by `patient_id = current_user`

Apart from having roles for the users, there will be an instance profile that provides an IAM role which will have permission to create log groups in CloudWatch logs, create log streams, and put log events using permissions of ‘logs:CreateLogGroup‘, ‘logs:CreateLogStream‘, and ‘logs:PutLogEvents‘ respectively. The instance profile will specifically not provide any permission to access RDS because database access can be provided at network and application levels by security groups and the connection string respectively.

4.3.6 DevSecOps Pipeline Design

The CI/CD pipeline is implemented in GitHub Actions and is triggered on every push to the ‘main‘ branch. The pipeline executes six stages in sequence. A failure at any stage with a critical finding terminates the pipeline and the deployment does not proceed.

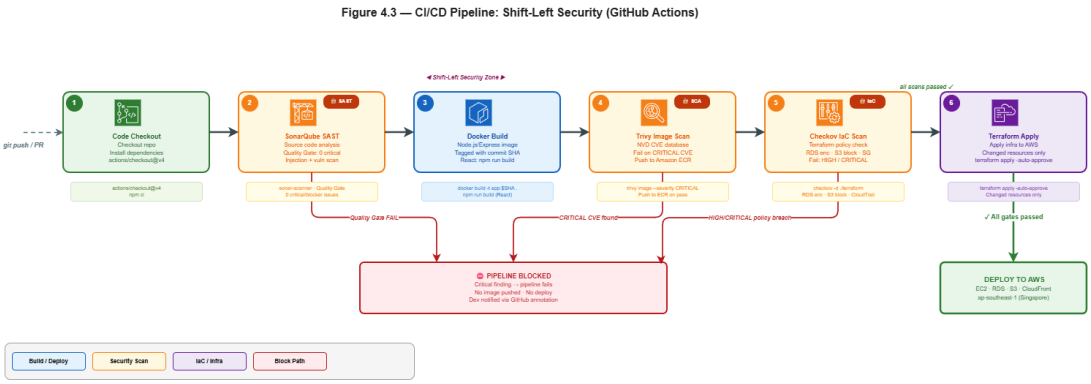


Figure 4.7 CI/CD Pipeline with Shift-Left Security Gates (GitHub Actions)

Table 4.10 DevSecOps Pipeline Stages Summary

Stage	Stage Name	Core Tool	Target Artifact	Failure / Gate Condition
1	Code Checkout	GitHub Actions	Source Code	Dependency resolution or fetch failure
2	SonarQube SAST	SonarQube Scanner	Application Code	Quality Gate failure (Critical/Blocker)
3	Build	Docker / npm	Container / Static	Compilation or image assembly failure
4	Trivy Image Scan	Trivy	Backend Image	Any `CRITICAL` severity CVE finding
5	Checkov IaC Scan	Checkov	Terraform Files	Any `HIGH` or `CRITICAL` policy violation
6	Terraform Apply	Terraform CLI	AWS Infrastructure	Cloud provider API or syntax execution error

4.3.6.1 Pipeline Stages Detailed Functional Description

Stage 1 Code Checkout: The application repository is cloned into the volatile workspace instance hosted by GitHub Actions. Project dependencies are resolved and cached to minimize subsequent run duration.

Stage 2 SonarQube SAST Scan: The source code is scanned statically to uncover structural defects, injection vulnerabilities, and quality issues. The pipeline enforces a strict baseline requiring zero unaddressed vulnerabilities matching or exceeding critical priority thresholds.

Stage 3 Build: The application elements undergo compilation and packaging. The Node.js and Express backend is bundled into an isolated Docker container image, tagged using the short Git commit SHA, while the React frontend generates optimized static production build output.

Stage 4 Trivy Image Scan: The generated application image is scanned against updated vulnerability registers. Discovering any unpatched vulnerability categorized as `CRITICAL` breaks the step. Clean images are exported to the Amazon Elastic Container Registry (ECR), static UI elements sync with Amazon S3, and the CloudFront distribution cache triggers invalidation.

Stage 5 Checkov IaC Scan: Infrastructure definitions are parsed for configuration vulnerabilities. Enforced controls explicitly check for data encryption on storage targets (Amazon RDS), restrictive external blockages on S3, secure ingress rule parameters on security groups, and operational audit trails (AWS CloudTrail).

Stage 6 Terraform Apply: Following validation across all testing gates, the state configuration updates the cloud infrastructure footprint on AWS. Execution is differential; only target resources with structural configuration shifts undergo updates.

4.3.7 Application Layer Class Design

Figure 4.4 presents the class diagram for the Node.js/Express application layer, showing the six data model classes and the two service/middleware classes that together form the backend of the system.

The six model classes (‘User‘, ‘Patient‘, ‘Doctor‘, ‘MedicalRecord‘, ‘Appointment‘, and ‘AuditLog‘) correspond to the database schema provided in Section 4.4. They represent their respective database tables in the application with static methods for table query and manipulation. The two service classes (‘AuthController‘ and ‘RBACMiddleware‘) are implemented at the request handler level. ‘AuthController‘ is responsible for managing the processes of logging in, logging out, and validating the JWT token. ‘RBACMiddleware‘ handles all incoming requests and ensures the caller meets RBAC requirements before reaching the handler.

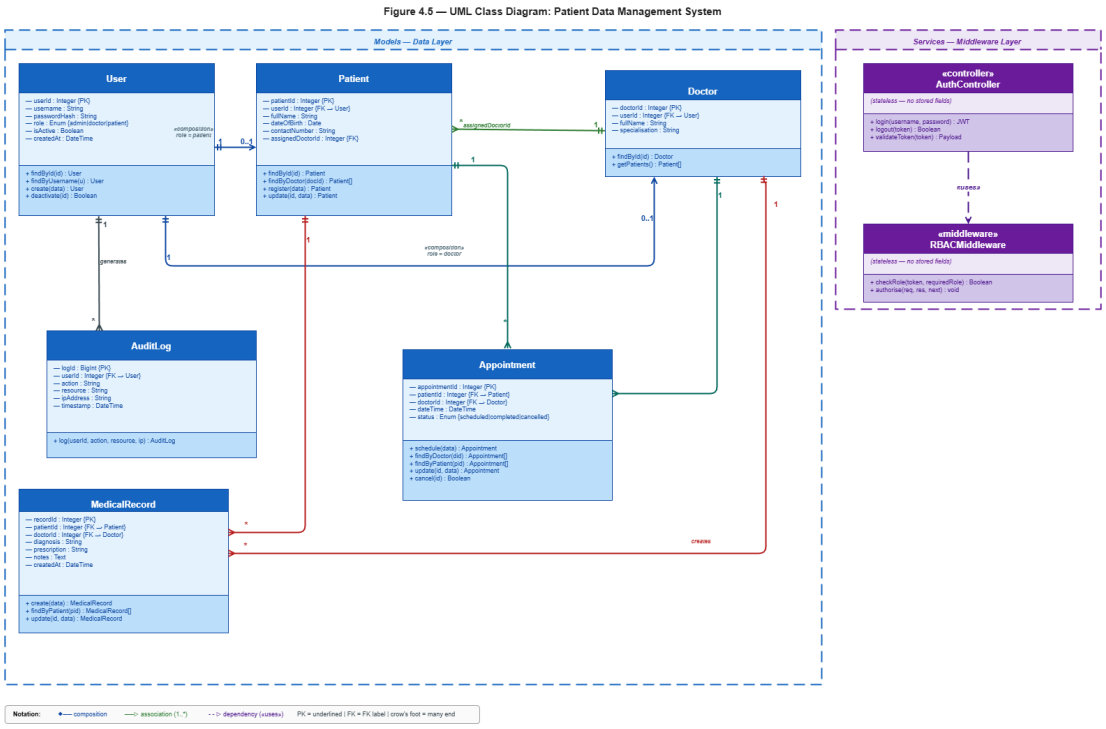


Figure 4.8 UML Class Diagram of the Patient Data Management System

4.3.8 Security Design

Security was one of the main design considerations from the get-go in the design of the system. The following describes the security measures put in place at each layer of the stack, namely: authentication, authorisation, API hardening, network segregation, data encryption, and audit monitoring, and aligns them with HIPAA Security Rule requirements.

4.3.8.1 Authentication Mechanism

Authentication identifies the user before access is granted to any system resources. The system uses a credentials-based process for authentication, whereby bcrypt hash passwords are used for authentication, while sessions are maintained using JWT.

Password security: Plaintext user passwords are not retained at any point. During user signup, the password undergoes the bcrypt hashing function with a cost of 12, resulting in a 60 character long hash value. A cost of 12 means that the verification

process takes around 250 milliseconds per hash on normal hardware, rendering offline brute force attack attempts computationally impossible. The plaintext password is deleted after the hashing process and never logged anywhere or written into any columns or error messages.

JWT Access Tokens: After a successful login process, the server generates a signed JWT that includes ‘userId‘, ‘role‘, and the time of expiration. This token is signed on the server-side using a secret key with the HMAC-SHA256 hashing function, making it impossible to forge on the client side. The access token expires in 15 minutes, whereas the refresh token lasts 7 days.

Cookie Security: The cookie containing the JWT will be set on the client side as an ‘httpOnly‘, ‘Secure‘, ‘SameSite=Strict‘ cookie. By setting ‘httpOnly‘ on the cookie, no JavaScript can access its value, mitigating the risk of XSS attacks which could steal the cookie from the client. By using the ‘Secure‘ cookie setting, only secure connections will be able to transfer the cookie from the client to the server.

Account Lockout Policy: Following the third unsuccessful login attempt, the flag ‘isActive‘ is set to false and all subsequent login attempts will fail with a standard message. The notification of the system alert is sent to the admin role. This policy fulfills the HIPAA security requirements of protection from repeated access attempts.

4.3.8.2 Authorization and Access Control

Authorisation decides what the authenticated user can perform. Authorisation has a two-tiered scheme, which comprises role-based access control (RBAC) in the application layer and row-level security (RLS) in the database layer.

Role-Based Access Control at Application Layer: The ‘RBACMiddleware‘ class captures every HTTP request sent to the API post-authentication. It fetches the ‘role‘ field value from the decoded JWT and matches it with the expected role on the corresponding route. In case the caller does not have the appropriate role assigned to him/her, the HTTP 403 Forbidden status is returned, and the request is not processed by the route handler.

Table 4.11 Role-Permission Matrix

Operation	Doctor	Admin	Patient
Login / Logout	✓	✓	✓
View assigned patients	✓	✓	—
Register / update patient	—	✓	—
Assign / reassign doctor	—	✓	—
Deactivate user account	—	✓	—
Create medical record	✓	—	—
View medical record	✓ (own patients)	—	✓ (own only)
Update medical record	✓ (own records)	—	—
Schedule appointment	—	✓	—
View appointments	✓	✓	✓ (own only)
Update / cancel appointment	—	✓	—

Row-Level Security (RLS) Database Layer: The policies defined using PostgreSQL RLS will enforce restrictions at the storage layer irrespective of any other layer. Hence, despite any circumvention of the application RBAC, such as by way of a hijacked API call, the database itself would not allow any retrieval of data rows that are unauthorized for the authenticated user. Important RLS policies are:

1. medical_records: a Doctor may only SELECT or UPDATE rows where doctor_id matches the session's userId. A Patient may only SELECT rows where patient_id maps to their own user account.

2. patients: a Doctor may only SELECT patients where assigned_doctor_id matches their doctorId.

3. appointments: visibility is scoped to the doctor_id or patient_id matching the session user.

Admin's database user account, on the other hand, operates under policies that grant full row-level permissions to all data in its administrative activities but are limited only to administration-related tables and activities.

4.3.8.3 API Security Controls

The REST API exposed by the Node.js/Express backend is hardened through a layered set of middleware controls applied globally to all routes.

Rate Limiting: With the help of ‘express-rate-limit‘, each request from an IP can be limited up to a total of 100 per 15 minutes. The limit for the login endpoint is reduced even further to just 10 in 15 minutes. All those requests which exceed the limit will get a 429 HTTP response status code.

Input Validity and Sanitization: All inputs to ‘POST‘ and ‘PUT‘ requests are validated by ‘express-validator‘ prior to execution of the corresponding route handlers. The validation process includes checking the input type, length, and other criteria, including formatting dates according to ISO 8601 and validating phone numbers as purely numeric values. If the input data fails the validation check, it is rejected with an HTTP 400 response and an informative message.

Parameterised Queries: All queries made to the database are done using the ‘node-postgres‘ library with parameterised queries. None of the queries have any form of SQL statement built up via concatenating user input together. User inputs for the parameterised query statements are passed to the PostgreSQL server where SQL injection is not possible.

Security Headers: The ‘Helmet.js‘ middleware package sets the following HTTP response headers on all API responses:

Table 4.12 HTTP Security Headers Configuration

Header	Value	Protection
`Content-Security-Policy`	`default-src 'self'`	Prevents inline script execution
`X-Content-Type-Options`	`nosniff`	Prevents MIME-type sniffing
`X-Frame-Options`	`DENY`	Prevents clickjacking via iframes
`Strict-Transport-Security`	`max-age=31536000`	Forces HTTPS for one year
`Referrer-Policy`	`no-referrer`	Suppresses referrer header leakage

CORS Policy: The CORS policy is set up such that any request must come from the cloud front origin of the clinic. Wildcards are strictly disallowed. The following methods are allowed for CORS policy: ‘GET‘, ‘POST‘, ‘PUT‘, and ‘DELETE‘.

Error Handling: Error handler middleware is used to capture any exceptions that have not been handled. The response to the user contains a simple error message together with an HTTP status code. Information such as stack trace, database error message, and server paths is never exposed since this might be harmful.

4.3.8.4 Network Security Controls

The AWS network architecture enforces isolation between public-facing, application, and database tiers using a layered set of controls that ensure no component is directly reachable unless explicitly permitted.

VPC and Subnet Isolation: The whole setup takes place within a private Virtual Private Cloud (VPC). Both EC2 and RDS servers are located in the private subnet and do not have any internet connectivity whatsoever. The Application Load Balancer (ALB) and the NAT Gateway are the only services in the public subnet. In other words, there is no way for anyone from the internet to get access to the application/database tiers unless going through the ALB.

Security Groups: AWS Security Groups act as stateful virtual firewalls at the instance level. Three security groups enforce least-privilege ingress rules:

Table 4.13 Network Security Group Configuration Matrix

Security Group	Inbound Rule	Source
`ALB-SG`	TCP 443 (HTTPS)	0.0.0.0/0
`EC2-SG`	TCP 3000 (Node.js)	`ALB-SG` only
`RDS-SG`	TCP 5432 (PostgreSQL)	`EC2-SG` only

Inbound traffic via SSH or any other administrative ports from the internet will not be allowed. Access to the EC2 instance for performing any kind of maintenance tasks can only be accomplished via AWS Systems Manager (SSM) Session Manager where no inbound ports are created.

ALB HTTPS Termination: The Application Load Balancer is provided with a listener on HTTPS port 443 that utilizes the SSL policy named 'ELBSecurityPolicy-TLS13-1-2-2021-06', which mandates the use of TLS 1.2 as the minimum protocol but also supports TLS 1.3. The other HTTP listener on port 80 issues an HTTP 301 redirection to its HTTPS counterpart URL. SSL certificates are taken care of by AWS Certificate Manager.

NAT Gateway: Outbound internet traffic from the EC2 instance is routed via the NAT Gateway located in the public subnet. The EC2 instance does not have an Elastic IP address or any outbound internet connection path; however, it still maintains the capacity for outgoing internet communications when necessary.

4.3.8.5 Data Encryption

All patient health information (PHI) is encrypted both at rest and in transit, satisfying the HIPAA encryption specification.

In-Transit Encryption: The traffic between the client browser and the system is encrypted using TLS 1.2 or above, terminating at the ALB. SSL encryption is used for the network traffic between the EC2 application server and the RDS PostgreSQL server, and it is enabled through the ‘rds.force_ssl‘ parameter value as ‘1‘. There is no plaintext communication link anywhere in the data path.

At-Rest Encryption Database: The RDS PostgreSQL instance is provisioned via ‘storage_encrypted = true‘ through Terraform, where encryption uses an AWS CMK (Customer Managed Key). All data files, automatic backups, read replicas, and database snapshots are encrypted using the same CMK. Automatic key rotation for the CMK takes place once per year. In other words, if the actual storage (EBS volume) is taken off, no one will be able to read anything without having access to the CMK.

Encryption at Rest Object Storage: The React static site build stored on the S3 bucket is protected with encryption via server-side encryption (SSE-S3 [AES-256]). The settings for block-public-acls, block-public-policy, ignore-public-acls, and restrict-public-buckets have all been configured to “true”, ensuring that the S3 bucket contents cannot be accessed directly from the Internet. The CloudFront distribution has access to the S3 bucket via the OAI (Origin Access Identity).

4.3.8.6 Monitoring, Audit Trail, and Incident Response

Continuous monitoring and a complete audit trail are required to detect security incidents and to demonstrate accountability for all actions taken on patient data.

Application Audit Log: All actions, including the creation, reading, updating, and deletion of data, done on patient records will be logged in the ‘audit_log‘ table through the ‘AuditLog‘ model class. These logs include the ‘userId‘, the type of action performed, the resource ID, the IP address, and timestamp. The logs form an immutable set of data for forensic purposes as per HIPAA audit controls requirements (§164.312(b)).

CloudTrail: CloudTrail is available in all AWS regions and logs each and every API request sent to the AWS account, including details such as who initiated it,

its IP, and the exact time it was executed. CloudTrail logs are uploaded to an S3 bucket having MFA delete set to ”on,” and this makes sure that these logs cannot be tampered with.

AWS CloudWatch: Application logs from the EC2 instance are streamed to CloudWatch Logs using the CloudWatch agent. CloudWatch alarms are configured for the following conditions:

1. Five or more failed login attempts within a five-minute window → triggers SNS notification to the administrator.

2. HTTP 5xx error rate exceeding 1% of requests → alarm for application layer failures.

3. RDS CPU utilisation exceeding 80% for five consecutive minutes → capacity alert.

AWS Security Hub: This service combines findings from AWS GuardDuty (threat detection), Amazon Inspector (vulnerability scanning for EC2 instances), and the Checkov IaC scan in the CI/CD pipeline into one security posture view. Any finding marked as either ‘HIGH‘ or ‘CRITICAL‘ triggers an alert.

Table 4.14 HIPAA Security Rule Compliance Mapping

HIPAA Requirement	**Reference**	**Implementation**
Access Control	§164.312(a)(1)	RBAC middleware + PostgreSQL RLS
Unique User Identification	§164.312(a)(2)(i)	UUID per user, bcrypt passwords
Automatic Logoff	§164.312(a)(2)(iii)	JWT 15-minute access token expiry
Encryption / Decryption	§164.312(a)(2)(iv)	KMS CMK at rest, TLS 1.2+ in transit
Audit Controls	§164.312(b)	`audit_log` table + AWS CloudTrail
Integrity Controls	§164.312(c)(1)	PostgreSQL FK constraints + RLS policies
Person or Entity Authentication	§164.312(d)	bcrypt cost 12 + account lockout after 3 failures
Transmission Security	§164.312(e)(1)	HTTPS-only ALB, `httpOnly` JWT cookie, TLS RDS connection

4.4 Database Design

4.4.1 Entity-Relationship Model

The database schema consists of six tables: ‘users‘, ‘patients‘, ‘doctors‘, ‘medicalrecords‘, ‘appointments‘, and ‘auditlog‘. Figure 4.4 presents the entity-relationship diagram.

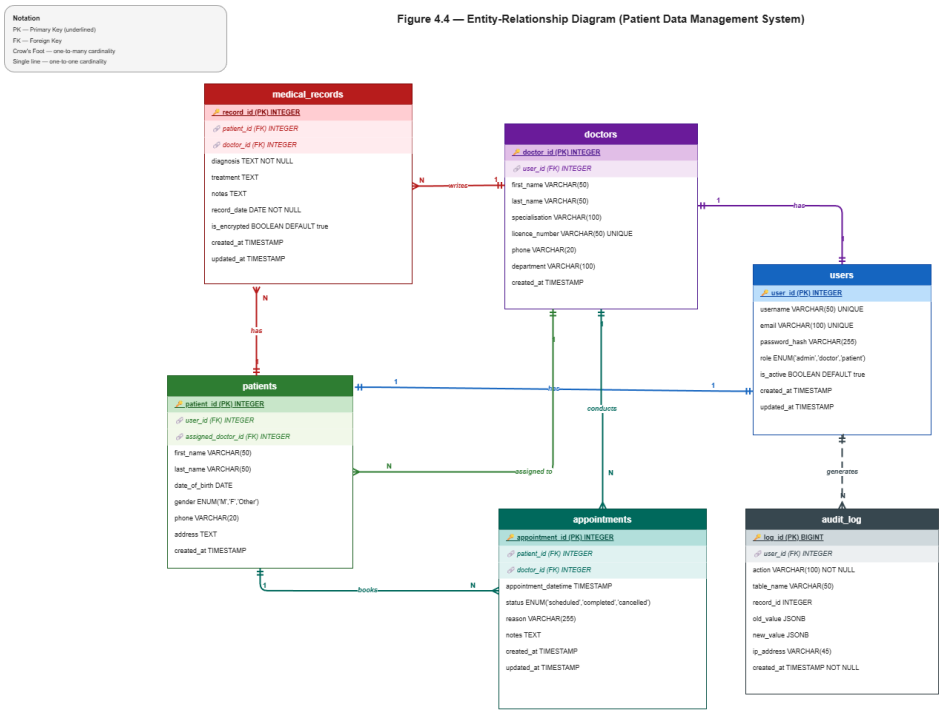


Figure 4.9 Entity-Relationship Diagram of the Patient Data Management System Database Schema

4.4.2 Database Schema

users: Stores authentication credentials and role assignment for all system users. Passwords are stored as bcrypt hashes — the plaintext password is never persisted.

Table 4.15 Database Table: `users`

Column	Type	Constraints	Description
`user_id`	`UUID`	PRIMARY KEY	Unique identifier
`username`	`VARCHAR(50)`	UNIQUE, NOT NULL	Login username
`password_hash`	`VARCHAR(255)`	NOT NULL	bcrypt hash of password
`role`	`ENUM('doctor', 'admin','patient')`	NOT NULL	Assigned role
`is_active`	`BOOLEAN`	DEFAULT TRUE	Account active status
`created_at`	`TIMESTAMP`	DEFAULT NOW()	Account creation timestamp
`last_login`	`TIMESTAMP`	NULLABLE	Last successful login

patients: Stores patient demographic information. Linked to the ‘users‘ table for authentication and to the ‘doctors‘ table for the assigned doctor relationship.

Table 4.16 Database Table: `patients`

Column	Type	Constraints	Description
`patient_id`	`UUID`	PRIMARY KEY	Unique identifier
`user_id`	`UUID`	FK → `users.user_id`	Authentication account
`first_name`	`VARCHAR(100)`	NOT NULL	Patient first name
`last_name`	`VARCHAR(100)`	NOT NULL	Patient last name
`date_of_birth`	`DATE`	NOT NULL	Date of birth
`contact_number`	`VARCHAR(20)`	NULLABLE	Phone number
`assigned_doctor_id`	`UUID`	FK → `doctors.doctor_id`	Assigned treating doctor
`registered_at`	`TIMESTAMP`	DEFAULT NOW()	Registration timestamp

doctors: Stores clinical staff information, linked to the ‘users‘ table for authentication.

Table 4.17 Database Table: `doctors`

Column	Type	Constraints	Description
`doctor_id`	`UUID`	PRIMARY KEY	Unique identifier
`user_id`	`UUID`	FK → `users.user_id`	Authentication account
`first_name`	`VARCHAR(100)`	NOT NULL	Doctor first name
`last_name`	`VARCHAR(100)`	NOT NULL	Doctor last name
`specialisation`	`VARCHAR(100)`	NULLABLE	Medical specialisation

medicalRecords: Stores clinical records created by doctors. Each record is linked to exactly one patient and one creating doctor.

Table 4.18 Database Table: `medical_records`

Column	Type	Constraints	Description
`record_id`	`UUID`	PRIMARY KEY	Unique identifier
`patient_id`	`UUID`	FK → `patients.patient_id`	Associated patient
`doctor_id`	`UUID`	FK → `doctors.doctor_id`	Creating doctor
`diagnosis`	`TEXT`	NOT NULL	Clinical diagnosis
`prescription`	`TEXT`	NULLABLE	Prescribed treatment
`notes`	`TEXT`	NULLABLE	Additional clinical notes
`created_at`	`TIMESTAMP`	DEFAULT NOW()	Record creation timestamp
`updated_at`	`TIMESTAMP`	DEFAULT NOW()	Last update timestamp

appointments: Stores scheduled appointments linking patients to doctors at a defined time.

Table 4.19 Database Table: `appointments`

Column	Type	Constraints	Description
`appointment_id`	`UUID`	PRIMARY KEY	Unique identifier
`patient_id`	`UUID`	FK → `patients.patient_id`	Attending patient
`doctor_id`	`UUID`	FK → `doctors.doctor_id`	Attending doctor
`scheduled_at`	`TIMESTAMP`	NOT NULL	Appointment date and time
`status`	`ENUM('scheduled', 'completed', 'cancelled')`	DEFAULT 'scheduled'	Current status
`notes`	`TEXT`	NULLABLE	Admin notes
`created_by`	`UUID`	FK → `users.user_id`	Admin who created booking

auditLog: An append-only table recording every significant data access event. This table satisfies the HIPAA audit control requirement and the CloudTrail complement at the application data level.

Table 4.20 Database Table: `audit_log`

Column	Type	Constraints	Description
`log_id`	`UUID`	PRIMARY KEY	Unique identifier
`user_id`	`UUID`	FK → `users.user_id`	User who performed the action
`action`	`VARCHAR(50)`	NOT NULL	Action type (CREATE, READ, UPDATE, DELETE)
`table_name`	`VARCHAR(50)`	NOT NULL	Target table
`record_id`	`UUID`	NOT NULL	Target record identifier
`ip_address`	`INET`	NULLABLE	Client IP address
`performed_at`	`TIMESTAMP`	DEFAULT NOW()	Event timestamp

4.4.3 Row-Level Security Policy

Row-level security on the PostgreSQL database is enabled for the ‘medicalRecords‘ table and ‘patients‘ table in order to provide data isolation within the database system, independent of application layer permissions. The following three policies are listed:

Policy 1 Doctor access to medical records: A doctor may only select, insert, or update records where ‘doctorId‘ matches the current database session user. Records assigned to other doctors are invisible and inaccessible.

Policy 2 Patient access to own records: A patient may only select rows from ‘medicalRecords‘ and ‘patients‘ where ‘patientid‘ matches their own user identifier. No patient can query another patient’s records regardless of application behaviour.

Policy 3 Admin exclusion from medical content: Admin database sessions are granted access to the ‘patients‘ and ‘appointments‘ tables only. Row-level security on ‘medicalRecords‘ denies all access to sessions authenticated with the admin role,

ensuring that administrative staff cannot view clinical record content even with direct database access.

This gives another level of protection; in case there is any vulnerability in the Node.js/Express application layer, the database will refuse to accept any queries that exceed their roles.

4.5 Interface Design

There are three unique interface views for the three types of users. These interface views have been made in such a way that there is role segregation, where all functionalities that a user needs to perform and can access are visible to him only.

4.5.1 Login Screen

All users have to log in through one login page. The login page displays two fields for the username and password entry along with the login button. After a successful login process, the user will be taken to the dashboard depending on the role obtained from the JWT token, such as Doctor Dashboard, Admin Dashboard, or Patient Portal.

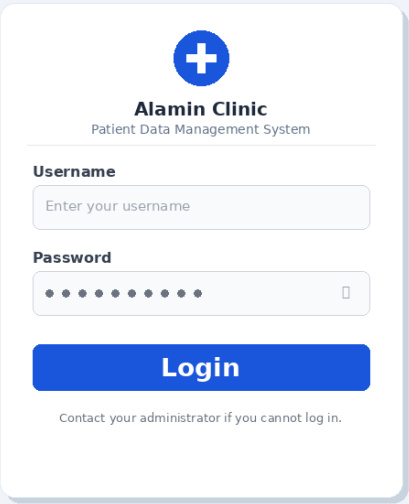


Figure 4.10 Login Screen Wireframe

4.5.2 Doctor Dashboard

The view that appears on Doctor Dashboard is a view where doctor's assigned patients appear along with appointments. The sidebar gives easy access to patient records and appointments.

The patients' list is comprised of the patient names along with the date of their latest visit and the View Records button. When a particular patient is selected, the system opens the patient details view page which shows the history of the medical records of the chosen patient and the New Record button.

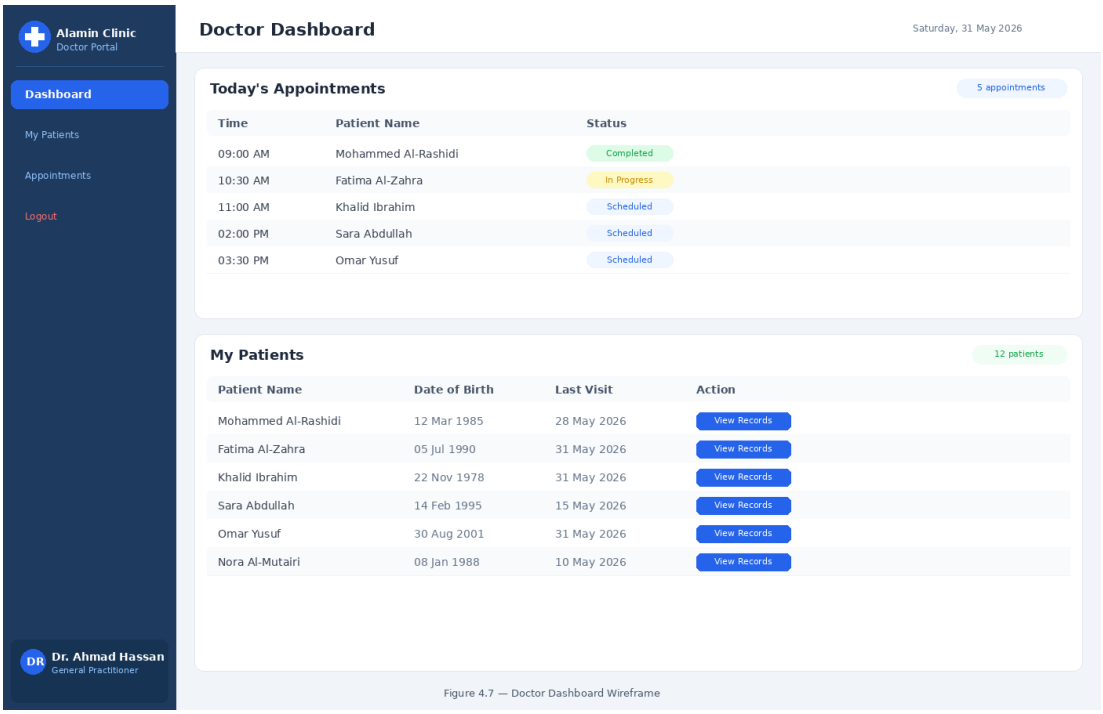


Figure 4.11 Doctor Dashboard Wireframe

4.5.3 Admin Dashboard

The Admin Dashboard centres on appointment management and patient registration. The navigation sidebar provides access to Patient Registration, Appointment Scheduling, and User Management.

Appointments Scheduling Screen has a calendar layout where one can select time slots. In the admin area, the user selects the patient, selects the doctor, chooses a day and time slot, and finally books it. Patient Registration Screen has a screen that allows filling of the patient's information.

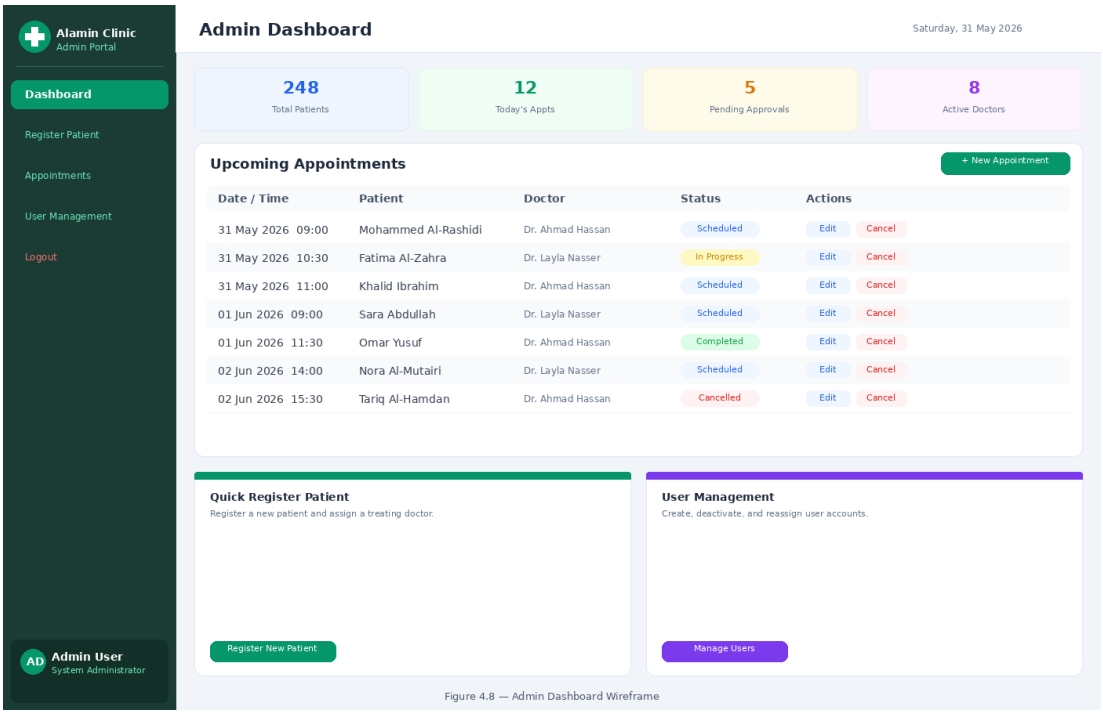


Figure 4.12 Admin Dashboard Wireframe

4.5.4 Patient Portal

Patient portal is a read-only portal. A patient can access his or her own medical records and appointments. There are no options to create, modify, or delete information within the patient's area.

In the records view, you will find a list of records in chronological order that show date, physician, and patient's diagnosis. When a record is clicked on, it gives you full information including any prescribed medication or additional notes from the doctor. In the appointments view, future appointments are listed by date and time.

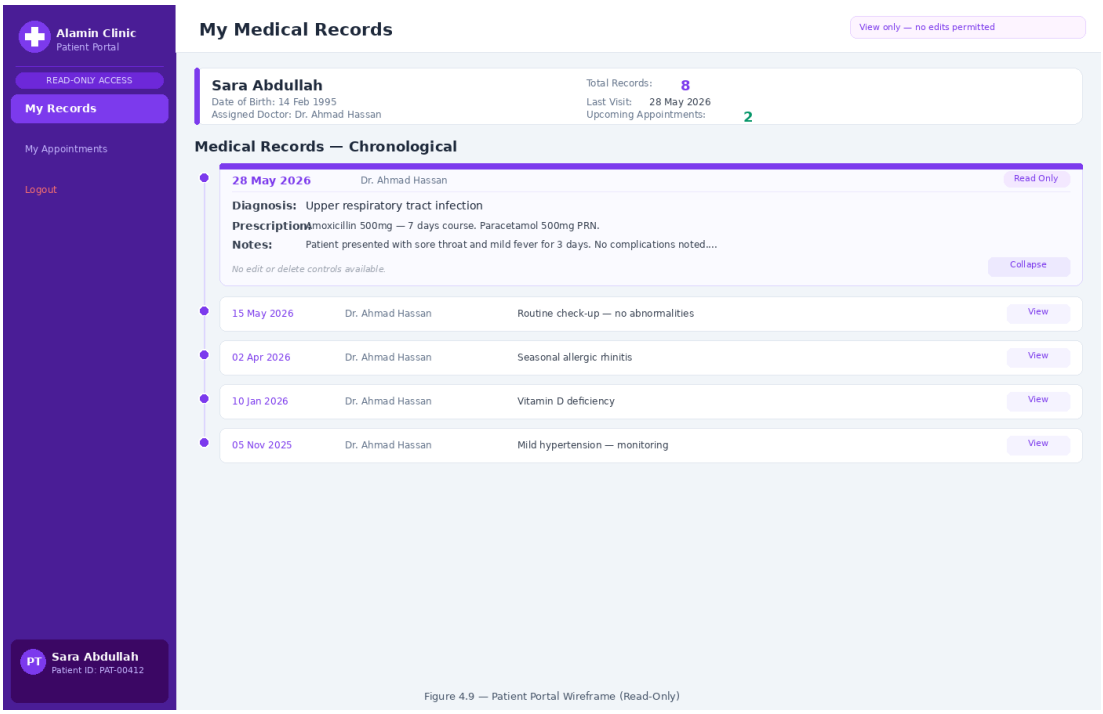


Figure 4.13 Patient Dashboard Wireframe

4.6 Chapter Summary

The chapter elaborates on the thorough requirement analysis and design of the Secure Cloud-Based Patient Data Management System, addressing the limitations outlined in the case study discussed in Chapter 2 and requirements described in Chapter 3. The architecture describes a very robust and resilient design, including the implementation of a three-tier architecture with six subnets on AWS VPC across two availability zones, along with layers of security groups, NACLs, and more precise IAM policies. In addition, a DevSecOps pipeline with automatic quality gates is introduced in order to ensure the left-shifted security approach to immediately block any deployment if it is found to be vulnerable. On the data layer, the PostgreSQL database employs a six-table schema with primary keys, foreign keys, an audit tracking table for HIPAA compliance, and separate RLS policy to ensure that role-based access control applies directly on the storage layer. Finally, the UI design defines the four different designs for the Login portal, Doctor Dashboard, Admin Dashboard, and Patient Portal, thus defining the architectural blueprint to support the implementation discussed in Chapter 5.

REFERENCES

Abbasi, N. and Smith, D. (2024). Cybersecurity in Healthcare: Securing Patient Health Information (PHI), HIPAA Compliance Framework and the Responsibilities of Healthcare Providers. *Journal of Knowledge Learning and Science Technology*. 3(3), 278–287. doi:10.60087/jklst.vol3.n3.p.278-287.

Ahuja, S., Mani, S. and Zambrano, J. (2012). A Survey of the State of Cloud Computing in Healthcare. *Network and Communication Technologies*. 1(2). doi: 10.5539/nct.v1n2p12.

Akbar, M. A., Smolander, K., Mahmood, S. and Alsanad, A. (2022). Toward Successful DevSecOps in Software Development Organizations: A Decision-Making Framework. *Information and Software Technology*. 147, 106894. doi: 10.1016/j.infsof.2022.106894.

Al-Issa, Y., Ottom, M. A. and Tamrawi, A. (2019). EHealth Cloud Security Challenges: A Survey. *Journal of Healthcare Engineering*. 2019, 1–15. doi: 10.1155/2019/7516035.

Argaw, S., Bempong-Ahun, N., Eshaya-Chauvin, B. and Flahault, A. (2019). The State of Research on Cyberattacks Against Hospitals and Available Best Practice Recommendations: A Scoping Review. *BMC Medical Informatics and Decision Making*. 19. doi:10.1186/s12911-018-0724-5.

Bass, L., Weber, I. and Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. SEI Series in Software Engineering. Pearson Education. ISBN 9780134049878. Retrievable at `https://books.google.com.my/books?id=fcwkCQAAQBAJ`.

Butt, A. U. R., Mahmood, T., Saba, T., Bahaj, S. A. O., Alamri, F. S., Iqbal, M. W. and Khan, A. R. (2023). An Optimized Role-Based Access Control Using Trust Mechanism in E-Health Cloud Environment. *IEEE Access*. 11, 138813–138826. doi:10.1109/ACCESS.2023.3335984.

Cobrado, U. N., Sharief, S., Regahal, N. G., Zepka, E., Mamauag, M. and Velasco, L. C. (2024). Access Control Solutions in Electronic Health Record Systems: A

Systematic Review. *Informatics in Medicine Unlocked*. 49, 101552. doi:10.1016/j.imu.2024.101552.

Espinha Gasiba, T., Andrei-Cristian, I., Lechner, U. and Pinto-Albuquerque, M. (2021). Raising Security Awareness of Cloud Deployments Using Infrastructure as Code Through CyberSecurity Challenges. In *Proceedings of the 16th International Conference on Availability, Reliability and Security*. ARES '21. New York, NY, USA: Association for Computing Machinery, 1–8. doi:10.1145/3465481.3470030.

Gaber, O., Anis Aziz, W. and Soliman, J. (2025). Three-Tier Cloud Application Deployment Using AWS With Identity Management. In *Proceedings of the International Conference on Cloud Computing and Services Science*.

Mendoza, C. and Reyes, C. (2023). Exploring the Impact of Shared Responsibility Models on Cloud Security Posture and Vulnerability Management. *Journal of Emerging Technologies*.

Paidy, P. and Chaganti, K. (2024). Resilient Cloud Architecture: Automating Security Across Multi-Region AWS Deployments. *International Journal of Emerging Trends in Computer Science and Information Technology*. 5(2), 82–93.

Rajapakse, R. N., Zahedi, M., Babar, M. A. and Shen, H. (2022). Challenges and Solutions When Adopting DevSecOps: A Systematic Review. *Information and Software Technology*. 141, 106700. doi:10.1016/j.infsof.2021.106700.

Ramayanam, S. (2025). Data Security and Compliance in Modernized Cloud-Enabled Healthcare and Financial Systems. *Journal of International Crisis and Risk Communication Research*, 406–414. doi:10.63278/jicrcr.vi.3378.

Shojaei, P., Vlahu-Gjorgievska, E. and Chow, Y.-W. (2024). Security and Privacy of Technologies in Health Information Systems: A Systematic Literature Review. *Computers*. 13(2), 41. doi:10.3390/computers13020041.

Singh, A. and Chatterjee, K. (2015). A Secure Multi-Tier Authentication Scheme in Cloud Computing Environment. In *2015 International Conference on Circuits, Power and Computing Technologies (ICCPCT-2015)*. 1–7. doi:10.1109/ICCPCT.2015.7159276.

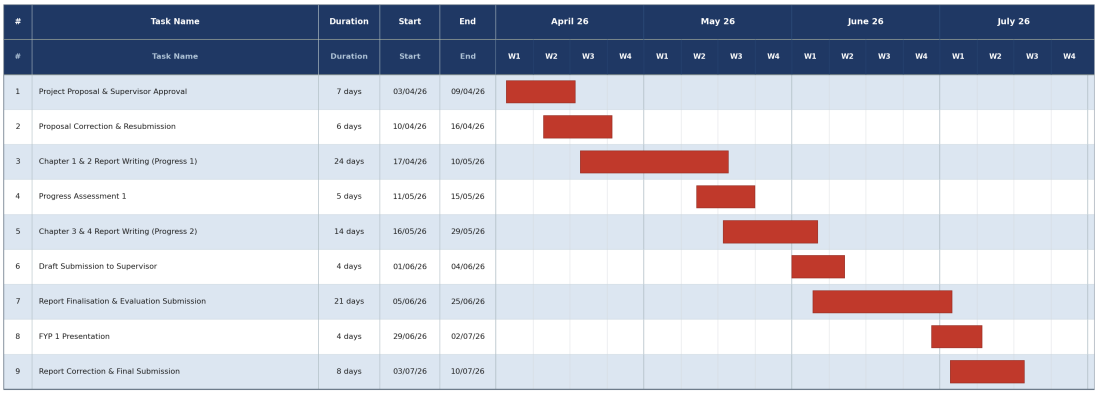
Thamer, N. and Alubady, R. (2021). A Survey of Ransomware Attacks for Healthcare Systems: Risks, Challenges, Solutions and Opportunity of Research. In *2021*

1st Babylon International Conference on Information Technology and Science (BICITS). 210–216. doi:10.1109/BICITS51482.2021.9509877.

Valdés-Rodríguez, Y., Hochstetter-Diez, J., Díaz-Arancibia, J. and Cadena-Martínez, R. (2023). Towards the Integration of Security Practices in Agile Software Development: A Systematic Mapping Review. *Applied Sciences*. 13(7), 4578. doi: 10.3390/app13074578.

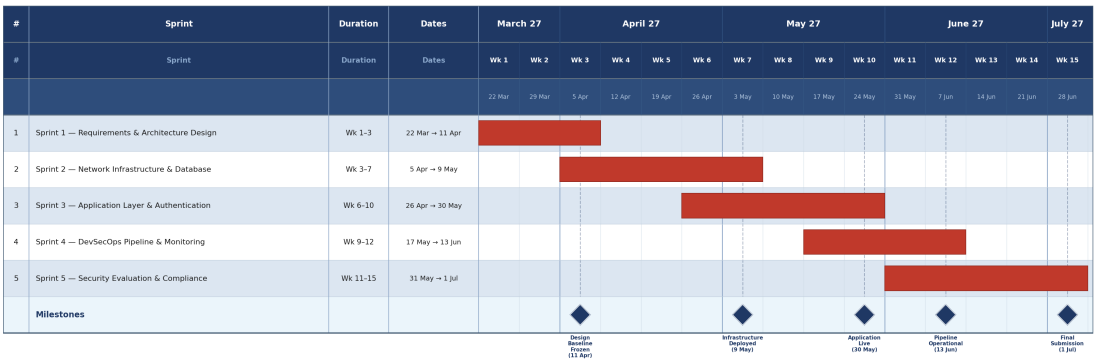
Verdet, A., Hamdaqa, M., Silva, L. and Khomh, F. (2023). *Exploring Security Practices in Infrastructure as Code: An Empirical Study*. doi:10.48550/arXiv.2308. 03952. ArXiv preprint arXiv:2308.03952.

Appendix A APPENDIX A



#	Task Name	Duration	Start	End
1	Project Proposal & Supervisor Approval	7 days	03/04/26	09/04/26
2	Proposal Correction & Resubmission	6 days	10/04/26	16/04/26
3	Chapter 1 & 2 Report Writing (Progress 1)	24 days	17/04/26	10/05/26
4	Progress Assessment 1	5 days	11/05/26	15/05/26
5	Chapter 3 & 4 Report Writing (Progress 2)	14 days	16/05/26	29/05/26
6	Draft Submission to Supervisor	4 days	01/06/26	04/06/26
7	Report Finalisation & Evaluation Submission	21 days	05/06/26	25/06/26
8	FYP 1 Presentation	4 days	29/06/26	02/07/26
9	Report Correction & Final Submission	8 days	03/07/26	10/07/26

Figure A.1 FYP 1 Gantt Chart



#	Sprint	Duration	Dates
1	Sprint 1 — Requirements & Architecture Design	Wk 1–3	22 Mar → 11 Apr
2	Sprint 2 — Network Infrastructure & Database	Wk 3–7	5 Apr → 9 May
3	Sprint 3 — Application Layer & Authentication	Wk 6–10	26 Apr → 30 May
4	Sprint 4 — DevSecOps Pipeline & Monitoring	Wk 9–12	17 May → 13 Jun
5	Sprint 5 — Security Evaluation & Compliance	Wk 11–15	31 May → 1 Jul
	Milestones		

Figure A.2 Figure 3.2 — FYP 2 Sprints Gantt Chart

Journal Articles

(a) Paper 1

(b) Paper 2

Conference proceedings

(a) Paper 3

(b) Paper 4